# Anwenderhandbuch

CODESYS Safety SIL2

Projektname: CODESYS Safety SIL2

Version: V8.0

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

# INHALT

Seite

**1 Einleitung ... 4**

1.1 Zielgruppe ... 4
1.2 Inhalt ... 4
1.1 Anwendungsbereich ... 4
1.2 Glossar ... 4
**2 Produktübersicht ... 6**

2.1 Funktionale Features ... 6
2.2 Safety Features ... 6
2.3 Projekteinstellungen ... 6
2.3.1 Compiler Optionen ... 6
2.3.2 Compiler Warnungen ... 6
2.4 Sicheres Logging ... 7
**3 Normen ... 7**

3.1 ISO / EN13849 / EN62061 ... 7
3.2 IEC61508 ... 7
3.3 IEC 61131-3 ... 8
3.4 Zusammenfassung ... 8
**4 Strukturierung ... 9**

4.1 Sichere Applikationsteile ... 9
4.1.1 Basic Level ... 9
4.1.2 Extended Level ... 10
4.1.3 System Level ... 10
4.2 Doppelte Ausführung ... 10
4.3 Zusammenfassung ... 11
**5 Programmierung ... 12**

5.1 Allgemein ... 12
5.1.1 Namenskonventionen ... 13
5.1.2 Aufrufkonventionen ... 13
5.1.3 Variablendeklarationen ... 13
5.1.4 Datentypen ... 14
5.1.5 Strukturierte Variablen ... 14
5.1.6 Tasks ... 15
5.1.7 Bibliotheken ... 16
5.1.8 Dokumentation ... 17
5.1.9 Ausdrücke ... 17
5.1.10 Pragmas ... 17
5.1.11 Objektorientierung ... 18
5.2 Strukturierter Text (ST) ... 18
5.2.1 Kontrollfluss ... 18
5.2.2 Ausdrücke ... 19
5.2.3 Operatoren ... 19
5.2.4 Mathematische Funktionen ... 20
5.3 Funktionsplan (FUP / FBD) und Kontaktplan (KOP / LD) ... 20
5.3.1 Kontrollfluss ... 20
5.3.2 Operatoren ... 20
5.3.3 Mathematische Funktionen ... 21
5.3.4 Standard Funktionsblöcke ... 21
5.4 Kontaktplan (KOP / LD) ... 21
5.5 Funktionsplan (FUP / FBD) ... 21
5.6 Continuous Flow Chart (CFC) ... 22
5.6.1 Allgemein ... 22
5.6.2 Kontrollfluss ... 22

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

5.6.3 Operatoren ... 23
5.6.4 Mathematische Funktionen ... 23
5.6.5 Standard Funktionsblöcke ... 23
5.7 UML Statechart (UML/SC) ... 24
5.7.1 Statechart Elemente ... 25
**6 Signalverarbeitung ... 27**
6.1 Entstehung und Verarbeitung sicherer Daten ... 27
6.2 Signalpegel ... 28
6.3 Verarbeitung nicht sicherer Daten ... 28
6.3.1 Compound Safety SIL2 ... 29
6.3.2 Non-Safe PRGs ... 29
6.4 FB Schnittstellen ... 30
6.5 Verarbeitung unsicherer E/As ... 31
6.6 CANopen Safety ... 31
6.6.1 Einsatz und Zertifizierung ... 31
6.6.2 Konfiguration ... 32
6.6.3 Diagnose ... 33
6.6.4 Limitierungen ... 34
6.6.4.1 Lizenzierung und Demo Modus ... 34
6.6.4.2 Global Failsafe Command ... 35
6.7 FSoEBase ... 35
6.7.1 Einsatz und Zertifizierung ... 35
6.7.2 Konfiguration ... 36
6.7.3 Limitierungen ... 36
6.7.3.1 Lizensierung und Demo Modus ... 36
6.7.3.2 Wiederanlauf und Fehleraufdeckung ... 37
**7 Anhang ... 38**
7.1 Abweichungen von PLCopen Safety ... 38
7.2 Abbildungsverzeichnis ... 39
7.3 Tabellenverzeichnis ... 40
7.4 Sicherheitsanforderungen ... 41
7.5 Literaturverzeichnis ... 43

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

# 1 Einleitung

## 1.1 Zielgruppe

Dieses Dokument richtet sich an Applikateure, die Sicherheitsgerichtete Steuerungen programmieren, sowie an OEM Kunden von 3S, welche sichere Funktionsbausteine bereitstellen.

Es wird vorausgesetzt, dass der Leser mit der Programmierung von IEC61131-3 Applikationen vertraut ist und Kenntnisse im Umgang mit CODESYS mitbringt. Außerdem ist ein Grundverständnis der Normen IEC61508, EN13849 und EN62061 beim Verständnis dieses Dokumentes hilfreich.

## 1.2 Inhalt

Dieses Dokument enthält Sicherheitsanforderungen, welche bei der Programmierung von CODESYS Safety SIL2 basierten Steuerungen zu beachten sind, sowie Programmierempfehlungen, welche die Zertifizierung einer Sicherheitsgerichteten Steuerung vereinfachen können.

Die genannten Programmierrichtlinien sind eine Ausformulierung der Programmierrichtlinien aus [TC5], mit wenigen Abweichungen und Verschärfungen. Die Abweichungen von [TC5] sind im Anhang 7.1 beschrieben.

## 1.1 Anwendungsbereich

Die Richtlinien in diesem Dokument setzen voraus, dass die Maschine nach EN13849 und/oder EN62061 zertifiziert wird. Liegt der Zertifizierung eine andere Norm zu Grunde, können die Klassifizierungen aus Kapitel 4 nicht unbedingt angewandt werden.

Dies bedeutet, dass für ein Projekt, welches nach einer anderen Norm als der EN13849 oder EN62061 zertifiziert wird, eigene Programmierrichtlinien verfasst werden müssen. Es wird jedoch empfohlen, sich auch in diesen Richtlinien, nach den in Kapitel 5 definierten Regeln zu richten, da diese Regeln auf bekannten Schwächen und Fehlerquellen der Sprachen basieren.

Zwingend erforderlich, ist die Einhaltung der in **Anhang 7.4** gelisteten **Sicherheitsanforderungen**.

Einzelne Softwareteile können stets abweichend von den Normen EN13849 oder EN62061, nach der IEC61508 zertifiziert werden. Dies entspricht dann dem in Kapitel 4.1.3 definierten "System Level".

**§H2-1.1 (Einhaltung der Sicherheitsanforderungen) [S3-4.2]**

| | |
|---|---|
| | Unabhängig von der, für die Applikation, angewandten Norm, bzw. des angewandten Zertifizierungslevels, müssen die in Anhang 7.4 gelisteten Sicherheitsanforderungen, bei der Programmierung einer Steuerung mit CODESYS Safety SIL2 eingehalten werden. |

## 1.2 Glossar

- **Basic Level**
  Definiert in Kapitel 4.1.1.
- **Debug Modus**
  Nach dem Start der Steuerung befindet sich diese zunächst im Safety Modus. Für die Wartung, oder einen Download, muss diese zunächst in den sog. Debug Modus versetzt werden.
- **E/E/PES**
  Elektrische, Elektronische und Programmierbar Elektronische Systeme. Eine SPS fällt in die Kategorie PES.
- **Extended Level**
  Definiert in Kapitel 4.1.2.
- **EVL**
  EVL steht für "External Variable List" und beschreibt eine globale Variablen Liste, bei welcher die Daten

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

im nicht sicheren Bereich liegen, um sie z.B. zwischen sicheren und nicht sicheren Programmen auszutauschen.

- **Non-Safe PRG**
  Der Begriff "Non-Safe PRG" bezeichnet ein Programm, welches nicht zertifizierten Code enthalten kann, jedoch rückwirkungsfrei ausgeführt wird.
- **FB**
  Funktionsbaustein nach IEC61131-3.
- **FBD**
  "Function Block Diagram" nach IEC61131-3.
- **FUP**
  "Funktionsplan" nach IEC61131-3.
- **FVL**
  "Full Variability Language" nach EN62061 und EN13849.
- **KOP**
  "Kontaktplan" nach IEC61131-3.
- **LVL**
  "Low Variability Language" nach EN62061 und EN13849.
- **Ladder**
  "Ladder" nach IEC61131-3.
- **Performance Level**
  "Performance Level" nach EN13849.
- **POU**
  "Programmable Object Unit" nach IEC61131-3.
- **Property**
  Die Bedeutung dieses Begriffes ist abhängig von seinem Kontext:
  **Bibliotheken:** Hier sind Properties, Benutzerdefinierte Eigenschaften der Bibliothek, welche bei den Projektinformationen hinterlegt werden können.
  **Applikation:** Hier sind Properties POUs, welche eine Set-/Get-Funktion für einen Variablenwert bereitstellen. Für den Anwender verhält sich ein Property wie eine Variable.
- **Ruhestromprinzip**
  Nach Ruhestromprinzip definiert ein High-Pegel (was in der Applikation einem TRUE entspricht) den Betriebszustand. Sind alle Signale auf Low befindet sich das System im sicheren Zustand.
- **SAFE-Datentypen**
  [TC5] definiert zu den Standarddatentypen einige neue, mit einem vorangestellten Präfix "SAFE" (z.B. SAFEBOOL zu BOOL). Diese sind funktional identisch zu den unsicheren Datentypen, erlauben der Entwicklungsumgebung und dem Reviewer jedoch sichere Daten einfacher zu identifizieren.
- **Safety Modus**
  Nach dem Start befindet sich die Steuerung zunächst im Safety Modus, und ist in der Lage eine funktional sichere Applikation auszuführen (siehe auch den Eintrag "Debug Modus" in diesem Glossar).
- **SIL**
  "Safety Integrity Level" nach IEC61508.
- **System Level**
  Definiert in Kapitel 4.1.3.
- **sichere FBs**
  Funktionsbausteine nach IEC61131-3, welche SAFE-Datentypen als Ein- bzw. Ausgang verwenden. Wenn diese Funktionsbausteine in einer sicheren Applikation verwendet werden sollen, müssen diese vorher, gemäß den Vorgaben aus Kapitel 4.1, validiert und/oder zertifiziert werden.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

# 2 Produktübersicht

Die hier vorliegende Beschreibung bezieht sich auf das Produkt CODESYS Safety SIL2. Dabei handelt es sich um ein Programmiersystem, welches die Programmierung Speicherprogrammierbarer Steuerungen nach IEC61131-3 erlaubt.

Bei dem Produkt CODESYS handelt es sich um ein Standard Programmiersystem, welches in vielen Bereichen eingesetzt wird. Darunter auch Bereiche, welche nicht sicherheitsgerichtet sind. Daher enthält das Produkt einige Features, welche nicht für die Verwendung in funktional sicheren Applikationen zugelassen sind.

## 2.1 Funktionale Features

Die tatsächlich unterstützten Features sind von dem Steuerungshersteller abhängig. Daher gelten die folgenden Regeln also nur, sofern nichts anderes von dem Steuerungshersteller definiert wurde.

**§H2-2.1 (Unsicheres Monitoring) [S2-4.7]**

Die in CODESYS Safety SIL2 zur Laufzeit angezeigten Variablen Werte werden über einen unsicheren Kommunikationskanal empfangen. Daher sind die Daten zunächst nicht sicher.

## 2.2 Safety Features

Anders, als andere mit CODESYS programmierbare Steuerungen, unterscheidet eine CODESYS Safety SIL2 Programmierbare Steuerung zwischen zwei Ausführungsmodi:

- **Safety Modus:**
  In diesem Modus befindet sich die Steuerung in der Regel nach dem Start. Möglicherweise unterstützt der Steuerungshersteller die Unterscheidung der unterschiedlichen Modi, was jedoch dem Handbuch der Steuerung zu entnehmen ist.
  In diesem Modus ist die Ausführung einer sicherheitsgerichteten Applikation erlaubt.
- **Debug Modus:**
  In diesen Modus, muss der Anwender explizit wechseln um einen Download zu machen, oder seine Applikation zu debuggen. Der Funktionsumfang im Debug Modus ist, im Vergleich zu dem Safety Modus, etwas erweitert und entspricht eher einer nicht sicheren Steuerung.
  In diesem Modus ist der sichere Betrieb einer sicherheitsgerichteten Applikation nicht gewährleistet.

## 2.3 Projekteinstellungen

In den Projekteinstellungen von CODESYS können projektspezifisch neben den Einstellungen die das Projekt betreffen auch Einstellungen vorgenommen werden, die Einfluss auf den Compiler haben. Nachfolgend werden nur die Einstellungen aufgelistet bei denen der Applikationsentwickler in der Verwendung eingeschränkt wird.

### 2.3.1 Compiler Optionen

| Einstellung | Erlaubter Wert |
|---|---|
| Compiler Version | Version des in dem CODESYS Safety SIL2 Setup enthaltenen, freigegebenen Compilers |
| Konstanten ersetzen | eingeschaltet |
| Compiler Warnungen | > 0 |
| Unicode-Zeichen für Bezeichner zulassen | ausgeschaltet |

### 2.3.2 Compiler Warnungen

Es wird empfohlen alle Warnungen zu aktivieren. Ausnahmen für deaktivierte Warnungen sollten von dem Applikationsentwickler ausreichend begründet werden.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

### 2.4 Sicheres Logging

Die Safety Seite im Device Dialog zeigt sichere Log Meldungen der Steuerung an. Die Meldungen werden über eine CRC gegen Modifikationen geschützt. Es können jedoch Daten verloren gehen wenn der Speicher für die Log Meldungen überläuft.

**§H2-2.2 (Sicheres Logging) [S2-4.18]**

Log Meldungen, welche auf der "Safety Page" beim Gerät angezeigt werden, sind über eine CRC abgesichert. Wurden die Daten während des Einlesevorgangs modifiziert, wird dies in der Oberfläche entsprechend angezeigt.

Innerhalb dieses Logs wird nur die Korrektheit, nicht jedoch die Vollständigkeit der Daten sichergestellt.

## 3 Normen

### 3.1 ISO / EN13849 / EN62061

Sicherheitsbezogene Maschinen, oder Steuerungen in solchen Maschinen, müssen häufig nach den Normen EN13849 oder EN62061 zertifiziert werden.

In Bezug auf die Software unterscheiden beide Normen zwischen zwei Klassen von Programmiersprachen:

- LVL: Limited Variability Language
  Programmiersprachen mit eingeschränktem Sprachumfang
- FVL: Full Variability Language
  Programmiersprachen mit uneingeschränktem Sprachumfang

Zu den LVLs zählen insbesondere die grafischen Sprachen der IEC61131-3, wie z.B. FUP oder KOP. In die Kategorie FVL fallen all jene textuellen Sprachen, wie C, ST, Assembler oder AWL, wobei prinzipiell alle Sprachen, die der IEC61508 genügen, erlaubt sind.

**Hinweis:** Bei entsprechender Limitierung des Sprachumfangs, kann auch eine textuelle Sprache wie ST als LVL klassifiziert werden.

Die EN13849 betrachtet das Gesamtsystem, und verweist für die Zertifizierung der Software auf die IEC61508 bzw. EN62061. Die EN62061 verweist in Kapitel 6.11.3.1.1 bei Verwendung einer Programmiersprache mit uneingeschränktem Sprachumfang, auf die IEC61508. Abhängig von der Komplexität der Software, findet somit die EN62061 oder die IEC61508 anwendung.

Da die Sicherheitslevel in den beiden Normen EN13849 und IEC61508 jedoch grundlegend anders definiert und benannt sind, gibt es dafür die Entsprechungstabelle **Tabelle 1**.

| PL | SIL |
|---|---|
| a | Keine Entsprechung |
| b | 1 |
| c | 1 |
| d | 2 |
| e | 3 |

**Tabelle 1** Entsprechung von PL zu SIL

Jegliche Software, die für eine Steuerung auf Performance Level d geschrieben wird, muss entsprechend **Tabelle 1**, nach SIL2 zertifiziert sein, wogegen für Performance Level c, z.B. bereits SIL1 ausreicht.

### 3.2 IEC61508

Die Norm IEC61508 beschreibt die Zertifizierung elektrischer, elektronischer und programmierbar elektronischer Systeme (kurz: E/E/PES). Nach dieser Norm können auch integrierbare Softwareteile zertifiziert werden.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

Sog. Harmonisierte Normen, wie die EN13849, verweisen bei der Zertifizierung von Software in weiten Teilen auf die IEC61508.

Die Software wird in der IEC61508 nach ihrem Integritätslevel klassifiziert. Die Integrität der Software lässt sich durch geeignete Coding Standards, entsprechende Diagnosefunktionen, sowie einer hohen Testabdeckung erhöhen.

### 3.3 IEC 61131-3

Die IEC61131-3 beschreibt die Programmiersprachen, die zur Programmierung von Steuerungen eingesetzt werden. In dieser Norm wird noch kein Fokus auf die funktionale Sicherheit gesetzt. Das hier vorliegende Dokument, beschäftigt sich allerdings überwiegend mit diesen Programmiersprachen und definiert alle Anforderungen spezifisch auf diese Sprachen zugeschnitten.

### 3.4 Zusammenfassung

Durch die Komplexität moderner Maschinen ist es schwierig diese komplett zu betrachten und abzunehmen. Stattdessen werden verschiedene Hard- und Softwareteile vorzertifiziert, sodass sich der Prüfer bei der Endabnahme der konfektionierten Maschine, auf die Betrachtung der wesentlichen Sicherheitsfunktionen konzentrieren kann.

Applikationen, die mit CODESYS Safety SIL2 für eine entsprechend geeignete Steuerung geschrieben werden, dürfen erst in sicherheitsgerichteten Umgebungen eingesetzt werden, wenn sie von der jeweils zuständigen Zertifizierungsstelle abgenommen wurden.

**§H2-3.1 (Abnahme der IEC Applikation) [S2-4.2]**

Eine IEC Applikation, welche funktionale Sicherheitsfunktionen implementiert, muss gemäß der für das Anwendungsgebiet geltenden Normen validiert / zertifiziert werden.

Alle innerhalb der Applikation verwendeten Bibliotheken müssen gemäß §H2-4.5 validiert oder zertifiziert worden sein.

- Verwendet die Applikation selbst ausschließlich LVLs, genügt für diese ein funktionaler Test.
- Verwendet die Applikation FVLs, müssen diese Applikationsteile vollständig über Modultests, gemäß IEC61508, getestet werden.

Ist für die Abnahme ein Review des Applikationscodes erforderlich, muss dieser manuell durchgeführt werden. Die Warnings des Compilers sowie der CODESYS Safety SIL2 Erweiterung können nur unterstützend herangezogen werden.

**§H2-3.2 (Review der IEC Applikation) [S2-4.17]**

Basierend auf den geltenden Sicherheitsbestimmungen für Maschinen, können Code Reviews erforderlich sein. Warnings des Compilers oder der Safety SIL2 Erweiterung im Programmiersystem dienen dabei lediglich als Unterstützung. Sie ersetzen allerdings keine Regel, die beim Review überprüft wird.

Wird die gleiche Applikation in mehreren Maschinen verwendet, ist sicher zu stellen, dass der Stand der reproduzierten Applikationen exakt dem Stand entspricht, welcher bei der Abnahme getestet und inspiziert wurde. Ein Hilfsmittel hierfür bietet CODESYS Safety SIL2 innerhalb der Safety Seite im Device Dialog. Hier kann der Anwender im laufenden Betrieb überprüfen, ob das generierte Maschinencode der Applikation auf dem Target zu dem Stand des aktuell geöffneten Projektes passt.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

**§H2-3.3 (Archivierung der IEC Applikation) [S2-4.16]**

Der Stand der Applikation, welcher bei der Abnahme getestet wurde, muss archiviert werden.

Wird der gleiche Applikationsstand auf mehrere andere Maschinen übertragen, ist darauf zu achten, dass das exakt gleiche Binärimage auf diese Maschinen übertragen wird.

Als zusätzliches Hilfsmittel bei der Überprüfung der Maschine bietet CODESYS Safety SIL2 die Möglichkeit die GUIDs innerhalb der "Safety Page" beim Gerät einzusehen und zu vergleichen. Dies ersetzt jedoch nicht die Vorsicht des Kunden bei der Reproduktion der Applikation.

Um die Abnahme der kompletten Maschine zu erleichtern und bereits vorzertifizierte Teile wiederverwenden zu können, wird empfohlen die Software in verschiedene Teile zu untergliedern, welche nach unterschiedlichen Normen und in unterschiedlicher Tiefe validiert, bzw. zertifiziert werden können:

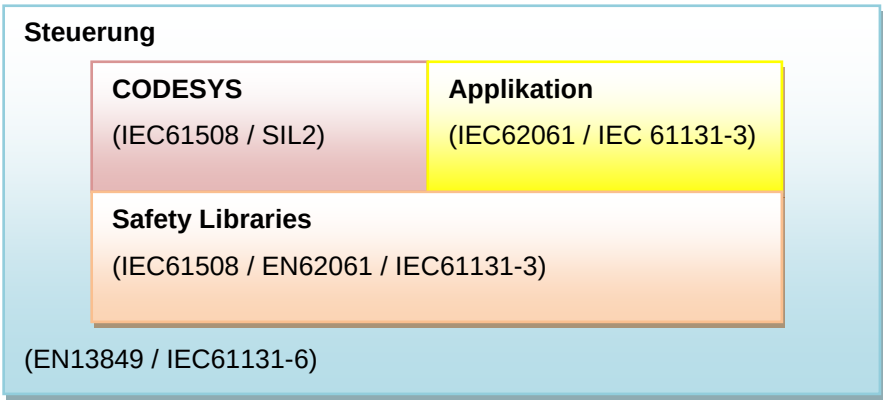



**Abbildung 1** Normenübersicht

# 4 Strukturierung

Gemäß [TC5], Kapitel 4.1, lassen sich sicherheitsgerichtete Applikationen in drei Teile untergliedern:

1. Basic Level
2. Extended Level
3. System Level

Die Programmiersprachen zur Applikationsentwicklung im Basic- und im Extended Level sind in [TC5] auf FUP und KOP (bzw. FBD und Ladder) limitiert.

Hierbei handelt es sich um Sprachen mit eingeschränktem Umfang (LVL), worunter nach EN62061 3.2.49 nur die grafischen Sprachen der IEC61131-3 fallen. Durch entsprechende Kodierrichtlinien kann allerdings nahezu jede Sprache als LVL klassifiziert, und somit im Basic- oder im Extended Level verwendet werden.

## 4.1 Sichere Applikationsteile

Als sichere Applikationsteile werden all jene bezeichnet, welche für die Ausführung der Sicherheitsfunktion benötigt werden. Prinzipiell also alle Routinen, die direkt sichere Ausgänge schreiben, oder sichere Eingänge für Funktionen- oder Bausteine bereitstellen, die dies tun.

Für solche sichere Applikationsteile wird die o.g. Klassifizierung aus [TC5] verwendet, welche nachfolgend beschrieben sind.

### 4.1.1 Basic Level

Eine sicherheitsgerichtete Applikation, welche nur aus vorzertifizierten Bausteinen besteht, welche auf einfache Weise miteinander verknüpft werden, fällt in die Kategorie "Basic Level". Diese Programme sind in den

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

Möglichkeiten der Verknüpfungen, sowie in der Art der Verwendung einzelner Bausteine so beschränkt, dass sie einfach lesbar bleiben und somit weniger Fehleranfällig sind.

Dieser Teil der Applikation wird später bei der Abnahme der Maschine im Integrationstest getestet. Bei moderater Komplexität einer Applikation im "Basic Level" müssen keine Modultests gemacht werden.

Die Programmierung ist auf LVLs beschränkt. Hierzu zählen in erster Linie FUP/FBD und KOP/Ladder. Mit entsprechender Limitierung des Sprachumfangs (siehe Kapitel 5.1.10) kann allerdings auch ST verwendet werden. Die Zertifizierung kann nach **EN62061** durchgeführt werden, ohne, dass auf die IEC61508 verwiesen wird.

### 4.1.2 Extended Level

Im "Extended Level" sind mehr Verknüpfungen erlaubt als im "Basic Level". Allerdings sind die Programmiersprachen immer noch auf die gleiche Auswahl wie im "Basic Level" limitiert.

Die Zertifizierung kann ebenfalls nach **EN62061,** ohne den Verweis auf die IEC61508, erfolgen. Allerdings ist der Aufwand bei der Zertifizierung, aufgrund der höheren Komplexität der Software, höher als im "Basic Level".

Extended Level Applikationsteile werden in der Regel in Form von Bibliotheken vertrieben. Diese Bibliotheken müssen das Property "SIL2" mit dem Wert TRUE gesetzt haben, damit der CODESYS Compiler bei der Verwendung im Basic Level keine Warnung ausgibt.

### 4.1.3 System Level

Der "System Level" beschränkt sich nicht mehr auf Sprachen mit eingeschränktem Umfang (LVLs), sondern erlaubt außerdem die Verwendung von Sprachen mit uneingeschränktem Umfang (FVLs). Die EN62061 verweist aufgrund der Verwendung von FVLs in Kapitel 6.11.3.1.1, bei der Zertifizierung auf die IEC61508.

In der Wahl der Programmiersprache ist der Entwickler nur durch die erforderliche Eignung nach IEC61508 eingeschränkt. Unter CODESYS Safety SIL2 im speziellen, bedeutet dies, dass der Kunde entweder einen beliebigen externen Compiler für extern implementierte Funktionsbausteine verwenden kann, oder eine der in Kapitel 5 genannten IEC61131-3 Sprachen.

Werden Applikationsteile auf System Level in Form von Bibliotheken verwendet, muss wie im Extended Level auch, das Property "SIL2" auf den Wert True gesetzt werden, damit der CODESYS Compiler keine Warnung bei der Verwendung ausgibt.

## 4.2 Doppelte Ausführung

Abhängig von der von dem Steuerungshersteller eingesetzten Hardware und Diagnosemitteln, kann es erforderlich sein, dass "sichere Applikationsteile" (s. Kapitel 4.1) doppelt berechnet werden müssen. Dies setzte voraus, dass alle Daten der sicheren Applikationsteile doppelt vorhanden sind, und diese doppelt ausgeführt werden.

Wenn dies bei der eingesetzten Steuerung notwendig ist, wird empfohlen alle sicheren Applikationsteile in Funktionsbausteinen zu kapseln, und diese doppelt zu instanziieren.

**§H2-4.1 (Doppelte Ausführung) [S2-4.4]**

Wird von dem Steuerungshersteller eine doppelte Ausführung des "sicherheitsgerichteten Codes", bzw. der "sicheren Applikationsteile" gefordert, müssen diese Applikationsteile doppelt ausgeführt werden.

Die statischen, sowie temporären Daten dieser Applikationsteile müssen dabei doppelt vorhanden sein, und bei beiden Aufrufen muss auf einer anderen Datenbasis gerechnet werden.

Ist eine doppelte Ausführung durch den Anwender gefordert, bedeutet dies auch, dass die Ein- und Ausgänge doppelt vorhanden sein müssen. Bei beiden Aufrufen muss dann auf jeweils unterschiedliche Ein- und Ausgänge zugegriffen werden.

Durch diese doppelt Ablage des I/O Abbildes, ist der I/O Treiber in der Lage die doppelt gerechneten Daten gegeneinander zu überprüfen.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

**§H2-4.2 (Doppeltes I/O Abbild) [S2-4.4]**

Wird von dem Steuerungshersteller eine doppelte Ausführung des "sicherheitsgerichteten Codes", bzw. der "sicheren Applikationsteile" gefordert, müssen alle sicheren Ein- und Ausgänge auch doppelt vorhanden sein.

Bei beiden Aufrufen des sicherheitsgerichteten Codes, muss dieser auf dem jeweils anderen Ein- bzw. Ausgang rechnen.

## 4.3 Zusammenfassung

Durch die Klassifizierung der Software in unterschiedlichen Komplexitätslevel (Basic, Extended, System), ist es möglich den Zertifizierungsaufwand je nach Komplexität des Softwareteils abzustufen und somit für bestimmte Teile zu vereinfachen.

Welche Validierungs- und Verifikationsmaßnahmen für den einzelnen Level genau gefordert werden, hängt von den zu erfüllenden Sicherheitsrichtlinien ab. Zur Entwicklung einer sicherheitsgerichteten Anwendung mit CODESYS Safety SIL2 sind jedoch zumindest Code Reviews erforderlich, um den Programmierrichtlinien und Sicherheitsanforderungen aus diesem Dokument zu genügen.

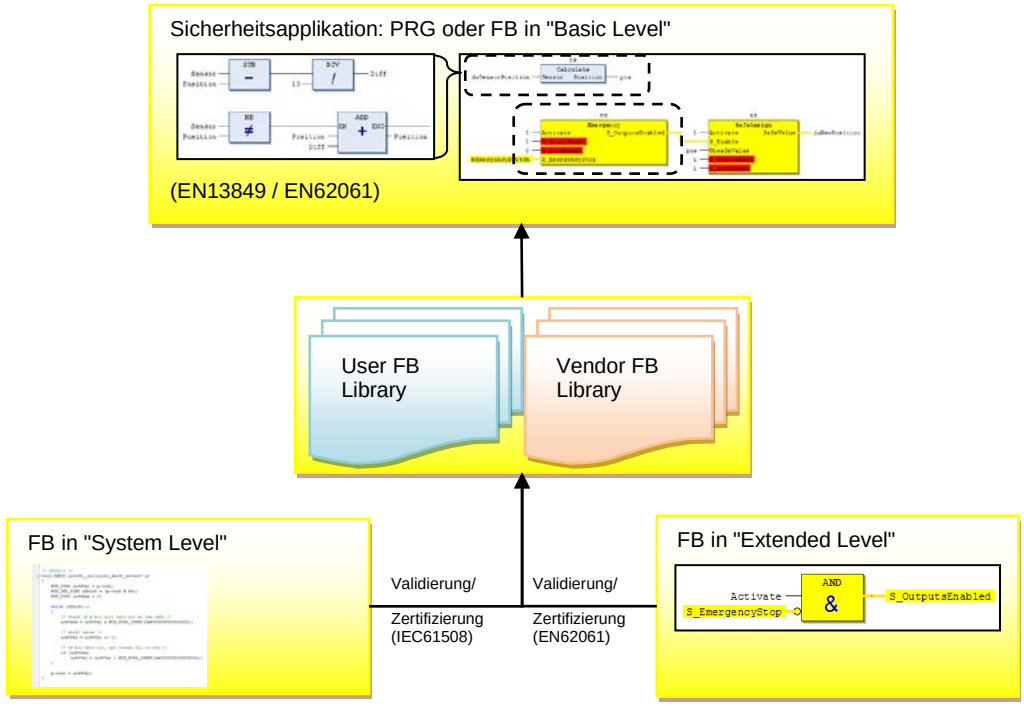



**Abbildung 2** Programmstruktur

**§H2-4.3 (Source Code Reviews) [S3-4.3]**

Um die Einhaltung der Sicherheitsanforderungen aus diesem Dokument sicherzustellen, sind Code Reviews für alle zu zertifizierenden Softwareteile erforderlich.

**§H2-4.4 (Warnungen im Code) [S2-5.1]**

Fehler und Warnungen, welche der CODESYS Compiler während der Kompilierung ausgibt sind ernst zu nehmen, da sie Auswirkungen haben können, welche für den Benutzer nicht zwangsläufig ersichtlich sind.

Daher muss sicherheitsgerichteter Code, welcher mit CODESYS entwickelt wurde, vollständig warnungsfrei sein.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

**§H2-4.5 (Verwendung von Bibliotheken) [S3-4.4], [S2-4.1]**

Werden bereits zertifizierte Bibliotheken verwendet, können diese gemäß ihrem Sicherheitshandbuch verwendet werden.

Falls von dem Hersteller der Bibliothek nicht anders vermerkt, können alle Bibliotheken, welche das Property "SIL2" auf True gesetzt haben, direkt in der Applikation verwendet werden.

Alle, Bibliotheken, welche das Property "SIL2 auf False stehen haben, dürfen nur aus unsicher angebundenen I/O Treibern heraus verwendet werden (dieses Interface wird idR. nur von dem Steuerungshersteller verwendet).

Alle Bibliotheken, die das Property "SIL2" nicht gesetzt haben, dürfen nicht direkt in der Applikation verwendet werden. Sie können jedoch wiederum von anderen, zertifizierten Bibliotheken verwendet werden.

In CODESYS Safety SIL2 kann zur Zertifizierung von POUs zwischen den nachfolgenden Leveln gewählt werden. Abhängig von dem gewählten Zertifizierungslevel, gelten dann andere, verschärftere Regeln und Limitierungen für die Programmierung.

1. **Basic Level**
   Sprachen: FUP, KOP, ...
   Zertifizierung: z.B. EN62061 / EN13849
2. **Extended Level**
   Sprachen: FUP, KOP, ...
   Zertifizierung: z.B. EN62061
3. **System Level**
   Sprachen: C, ST, Assembler, AWL, ...
   Zertifizierung: z.B. IEC61508

# 5 Programmierung

Dieses Kapitel enthält die Programmierrichtlinien für alle Sprachen der IEC61131-3 die unter CODESYS Safety SIL2 für sichere Applikationen eingesetzt werden dürfen. Die Limitierung auf diese Sprachen gilt für die gesamte Applikation, und ist unabhängig von der angewandten Norm. Die Regeln sind jedoch nur Empfehlungen, welche es ermöglichen die Zertifizierung gemäß [TC5] zu vereinfachen.

**§H2-5.1 (Erlaubte Programmiersprachen) [S3-4.5]**

Die erlaubten Programmiersprachen zur Programmierung einer CODESYS Safety SIL2 Steuerung, sind in Kapitel 5 gelistet. Beim Einsatz einer weiteren Sprache, ist ein getrennter Nachweis über die Eignung des eingesetzten Compilers vorzulegen.

Nicht alle hier definierten Regeln finden für jede, in Kapitel 4 definierten, Kategorie Anwendung. Ob eine Regel für eine Kategorie gültig ist, definiert das "X" in der Spalte zu der jeweiligen Kategorie.

**Anm.:** Um den Review eines Codes durchzuführen, müssen nur die Regeln betrachtet werden, die bei der Kategorie des Codes ein "X" haben. Zu einigen Schlüsselwörtern sind in einer Sprache keine Regeln definiert. Diese Zeilen sind mit '---' gekennzeichnet, was bedeutet, dass für diese Schlüsselwörter keine besondere Limitierung gilt.

Template: Vorlage_tecdoc_V1.0.dotx

## 5.1 Allgemein

© 3S-Smart Software Solutions GmbH

### 5.1.1 Namenskonventionen

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| S_ | Sichere Input- und Output-Variablen in FBs ohne den Prefix S_ an erster Stelle sind nicht erlaubt | X | X | |
| | Sichere, explizit definierte, lokale Variablen ohne den Prefix S_ an erster Stelle sind nicht erlaubt | X | X | |
| BOOL | BOOL Variablen ohne den Prefix x* sind nicht erlaubt | X | | |
| BYTE | BYTE Variablen ohne den Prefix by* sind nicht erlaubt | X | | |
| WORD | WORD Variablen ohne den Prefix w* sind nicht erlaubt | X | | |
| DWORD | DWORD Variablen ohne den Prefix dw* sind nicht erlaubt | X | | |
| LWORD | LWORD Variablen ohne den Prefix lw* sind nicht erlaubt | X | | |
| SINT | SINT Variablen ohne den Prefix si* sind nicht erlaubt | X | | |
| USINT | USINT Variablen ohne den Prefix usi* sind nicht erlaubt | X | | |
| INT | INT Variablen ohne den Prefix i* sind nicht erlaubt | X | | |
| UINT | UINT Variablen ohne den Prefix ui* sind nicht erlaubt | X | | |
| DINT | DINT Variablen ohne den Prefix di* sind nicht erlaubt | X | | |
| UDINT | UDINT Variablen ohne den Prefix udi* sind nicht erlaubt | X | | |
| LINT | LINT Variablen ohne den Prefix li* sind nicht erlaubt | X | | |
| ULINT | ULINT Variablen ohne den Prefix uli* sind nicht erlaubt | X | | |
| REAL | REAL Variablen ohne den Prefix r* sind nicht erlaubt | X | | |
| LREAL | LREAL Variablen ohne den Prefix lr* sind nicht erlaubt | X | | |

### 5.1.2 Aufrufkonventionen

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| FBs | Funktionsbausteininstanzen dürfen nicht mehrmals in einem Zyklus aufgerufen werden | X | X | X |
| | Funktionsbausteininstanzen müssen in jedem Zyklus aufgerufen werden | X | | |

### 5.1.3 Variablendeklarationen

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| VAR | --- | | | |
| VAR_INPUT/ VAR_OUTPUT | --- | | | |
| VAR_IN_OUT | IN/OUT Variablen sind nicht erlaubt | X | X | |
| VAR_GLOBAL/ | Globale Variablen sind nicht erlaubt | X | | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| EXTERNAL | Globale Variablen auf FB Ebene sind nicht erlaubt | | X | |
| CONSTANT | --- | | | |
| RETAIN | Retain Variablen sind nicht erlaubt | X | X | |

### 5.1.4 Datentypen

**§H2-5.2 (Gleitkomma Operationen) [S3-4.1]**

Gleitkommaoperationen sind im Basic- und im Extended Level nur erlaubt, wenn dies der Steuerungshersteller explizit so definiert. Im System Level obliegt die Risikoeinschätzung bei der Verwendung von Gleitkommaoperationen ohnehin dem Applikateur.

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| SAFE* | Sichere Daten, die nicht mit einem SAFE-Datentyp deklariert wurden, sind nicht erlaubt | X | X | |
| BOOL, BIT | --- | | | |
| BYTE, WORD, DWORD | --- | | | |
| SINT, USINT | Variablen diesen Typs sind nicht erlaubt (Konstanten sind erlaubt) | X | | |
| INT, UINT | Variablen diesen Typs sind nicht erlaubt (Konstanten sind erlaubt) | X | | |
| DINT, UDINT | Variablen diesen Typs sind nicht erlaubt (Konstanten sind erlaubt) | X | | |
| REAL, LREAL | Variablen diesen Typs sind nicht erlaubt (Konstanten sind erlaubt) | X | | |
| | Variablen diesen Typs sind nicht erlaubt, sofern sie nicht explizit vom Steuerungshersteller erlaubt wurden | X | X | |
| TIME, TOD, DATE, DT | Variablen diesen Typs sind nicht erlaubt (Konstanten sind erlaubt) | X | | |
| STRING | Verwendung als Variable oder Konstante ist nicht erlaubt | X | X | |
| ANY_* | Verwendung als Variable oder Konstante ist nicht erlaubt | X | X | |
| Konstanten | Ungetypte Konstanten sind nicht erlaubt | X | X | |

### 5.1.5 Strukturierte Variablen

**§H2-5.13 (Nicht natürlich alignte Strukturen) [S3-4.16]**

Verwendet eine POU eine nicht alignte Struktur, muss diese einem vollständigen Modultest unterzogen werden.

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| ARRAY | Arrays sind nicht erlaubt | X | | |
| | Arrays sind ohne explizite Bereichsprüfung nicht erlaubt | | X | |
| STRUCT | Strukturen sind als Interface zum Datenaustausch zweier Systeme nicht erlaubt | X | | |
| | Strukturen, welche nicht natürlich aligned sind, sind zum Datenaustausch zweier Systeme nicht erlaubt | | X | |
| ENUM | Arithmetische Operationen mit Enumerationen sind nicht erlaubt | X | X | |
| POINTER / REFERENCE | Pointer und Referenzen sind nicht erlaubt | X | X | |

### 5.1.6 Tasks

Um Probleme aufgrund von Nebenläufigkeiten in I/O Treibern zu vermeiden, wird die Verwendung einer Treiberinstanz auf eine Task beschränkt, sofern der Treiberhersteller die Verwendung nicht anderweitig erlaubt hat.

**§H2-5.3 (Multitasking und I/Os) [S2-4.5]**

Sofern nicht anders von dem Treiberhersteller erlaubt, darf eine I/O Treiber Instanz nur aus einer Task heraus verwendet werden. Dies bedeutet, dass I/Os eines Gerätes, oder eines Feldbusses nur aus einer Task heraus verwendet werden dürfen, und dass der Buszyklus (falls erforderlich) auch von dieser Task angestoßen werden muss.

**Anm.:** Sofern dieses Feature von der Steuerung angeboten wird, kann das I/O Mapping über die Registerkarte "Task-Aufstellung" in der Gerätekonfiguration, überprüft werden.

**§H2-5.4 (Mehrfachmappings von I/Os) [S2-4.6]**

Jeder I/O Kanal darf nur auf eine Variable gemappt werden. Mehrfaches Mapping von Teilen des Kanals, ist nicht erlaubt.

**Ausnahme:** Strukturierte Datentypen, welche nur in Teilen gemappt werden können. Bei Strukturierten Datentypen, gilt jedoch die gleiche Regel in Bezug auf ihre Struktur-Elemente.

**Bsp.:**

- Das Mapping des High- und des Low- Words von einem I/O Kanal vom Typ DWORD auf unterschiedliche Variablen ist nicht erlaubt.
- Das Mapping von einzelnen Bits aus einem I/O Kanal auf unterschiedliche Variablen ist nicht erlaubt.

Template: Vorlage_tecdoc_V1.0.dotx

| **Schlüsselwort** | **Regel** | **basic** | **extended** | **system** |
|---|---|---|---|---|
| Synchronisation | Zugriff auf Ein- und Ausgänge eines Gerätes oder Feldbusses aus mehreren Tasks ist nicht erlaubt | X | X | |
| | Zugriff auf die gleiche globale Variable, aus mehreren Tasks heraus ist nicht erlaubt | X | | |
| Zyklisch | Eine zyklische Task ohne definierten Watchdog ist nicht erlaubt | X | X | |
| | Eine Watchdogzeit, größer als die Zykluszeit der Task ist nicht erlaubt | X | X | |

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| | Eine Watchdog-Sensitivity, größer 1 ist nicht erlaubt | X | X | |
| Event | Eine Event-Task ohne definierten Watchdog ist nicht erlaubt | X | | |
| | Eine Event-Task mit undeterministischer, maximaler Zykluszeit ist nicht erlaubt | X | X | |
| Freewheeling | Eine Freewheeling-Task ohne definierten Watchdog ist nicht erlaubt | X | | |
| | Eine Freewheeling-Task mit undeterministischer, maximaler Zykluszeit ist nicht erlaubt | X | X | |

### 5.1.7 Bibliotheken

In einer sicherheitsgerichteten Applikation, welche mit CODESYS Safety SIL2 programmiert wird, ist nur die Verwendung speziell gekennzeichneter Bibliotheken erlaubt. Hierzu muss die Bibliothek das Property "SIL2" vom Typ BOOL gesetzt haben.

- Hat dieses Property den Wert True, kann die Bibliothek uneingeschränkt in einer sicherheitsgerichteten Applikation verwendet werden.
- Hat dieses Property den Wert False, kann die Bibliothek zwar eingebunden werden, darf allerdings nur von speziellen, unsicher angebundenen I/O Treibern verwendet werden.
- Ist dieses Property in einer Bibliothek nicht vorhanden, darf die Bibliothek nicht in einer sicherheitsgerichteten Applikation verwendet werden.

**Anm.:** Bibliotheken, welche das "SIL2" Property gesetzt haben, dürfen ihrerseits auch Bibliotheken verwenden, welche dieses Property nicht definiert haben.

**Anm.:** Hierdurch ist es z.B. möglich eine Teilfunktionalität aus einer fremden Bibliothek zu validieren und über eine eigene Bibliothek anzubieten ohne die fremde Bibliothek zu modifizieren.

**§H2-5.5 (Erlaubte Bibliotheken) [S3-4.4], [S2-4.1]**

In der Applikation dürfen nur Bibliotheken verwendet werden, welche das Property "SIL2" gesetzt haben.

Innerhalb von Bibliotheken, können hingegen auch andere Bibliotheken verwendet werden, welche ausreichend validiert wurden.

Wird in der Applikation eine Bibliothek ohne das SIL2 Property verwendet, führt dies zu einer Warnung beim kompilieren. Zu Debugzwecken kann diese Warnung durchaus ignoriert werden. Selbst die Verwendung von Firmwarefunktionalität der Steuerung, welche im Safety Modus nicht erlaubt sind, ist im Debug Modus möglich.

**Anm.:** Verwendet die Applikation unerlaubte Firmwarefunktionen, funktioniert dies zwar im Debug Modus, führt beim Start im Safety Modus jedoch zu einer Exception.

**§H2-5.6 (Validierung von Bibliotheken) [S3-4.6]**

Damit eine Bibliothek als bereits validierte Bibliothek innerhalb einer Applikation verwendet werden kann, muss die Verifikation auf der Zielhardware mit den gleichen Compilereinstellungen, mit welchen die Applikation später läuft, durchgeführt werden.

Um sicherzustellen, dass die Compilereinstellungen gleich sind, genügt es die gleiche Device Description für das Gerät zu verwenden, welche auch in der produktiven Maschine eingesetzt wird.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

### 5.1.8 Dokumentation

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| POU / FB Dokumentation | POU ohne eine sichtbare Beschreibung der funktionalen Spezifikation ist nicht erlaubt | X | X | X |
| | Eine Library ohne die folgenden Informationen innerhalb ihrer "Projektinformation" ist nicht erlaubt:<br>Autor, Erstellungsdatum, Release Datum, Version, Versionsgeschichte | X | X | X |
| Code Dokumentation | Code, welcher nicht angemessen dokumentiert ist, ist nicht erlaubt | X | X | |

### 5.1.9 Ausdrücke

**§H2-5.14 (Verwendung der Operatoren AND_THEN / OR_ELSE) [S3-4.17]**

Verwendet eine POU die Operatoren AND_THEN oder OR_ELSE besteht eine erhöhte Gefahr von möglichen Seiteneffekten. Aus diesem Grund müssen diese POUs einem vollständigen Modultest unterzogen werden.

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Typ-konvertierung | Implizite Typkonvertierungen innerhalb von Ausdrücken sind nicht erlaubt<br>**Ausnahme:** Die Operatoren AND und OR bei der Konvertierung von BOOL nach SAFEBOOL | X | X | |
| | Explizite Typkonvertierungen sind nicht erlaubt<br>**Ausnahme:** SAFE*_TO_*() | X | | |
| AND_THEN / OR_ELSE | Die Operatoren AND_THEN und OR_ELSE sind nicht erlaubt | X | | |

### 5.1.10 Pragmas

**§H2-5.15 (Verwendung von Pragmas) [S3-4.15]**

Pragmas sind nur erlaubt, wenn die betroffenen POUs einem vollständigen Modultest unterzogen wurden. Dies gilt zum einen für Pragmas innerhalb der POU aber auch für Pragmas innerhalb von Datentypen (z.B. Strukturen), welche in der POU verwendet werden.

Enthält eine Struktur solche kritischen Pragmas, muss dies in der Beschreibung der Struktur ersichtlich sein.

**Ausnahme:** Von dieser Regel ausgenommen ist das folgende Pragma:
`{attribute 'location':='16#800'}`

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| Pragmas | Pragmas innerhalb einer POU sind nicht erlaubt<br>**Ausnahme:**<br>`{attribute 'location':='16#800'}` | X | | |
| | Pragmas innerhalb einer Struktur sind nicht erlaubt<br>**Ausnahme:**<br>`{attribute 'location':='16#800'}` | X | | |

### 5.1.11 Objektorientierung

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| EXTENDS | Definition von Funktionsbausteinen, welche andere erweitern, ist nicht erlaubt | X | | |
| IMPLEMENTS | --- | | | |
| Mehrfach-Vererbung | ---<br>**Anm.:** CODESYS unterstützt Mehrfachvererbung ohnehin nur für Interfaces | | | |
| FB_Init / FB_Exit | Expliziter Aufruf von FB_Init() oder FB_Exit der Basisklasse, ist nicht erlaubt | X | X | X |
| | FB_Init und FB_Exit dürfen sich nicht auf einen einmaligen Aufruf verlassen.<br>Die beiden Funktionen müssen so robust implementiert sein, dass sie einen doppelten Aufruf der Funktionen überstehen. | X | X | X |

## 5.2 Strukturierter Text (ST)

Nach EN62061 wird ST nicht als LVL, sondern explizit als FVL klassifiziert. Durch die Kodierrichtlinien aus diesem Kapitel für Basic- und Extended Level wird der Umfang der Sprache jedoch soweit eingeschränkt, dass sie als LVL klassifiziert werden kann.

**§H2-5.7 (Strukturierter Text als LVL) [S3-4.7]**

| | |
|---|---|
| | Damit die Sprache Strukturierter Text als LVL klassifiziert werden kann, müssen die Programmierregeln aus Kapitel 5.1.10 für Basic- oder Extended Level eingehalten werden. |

Template: Vorlage_tecdoc_V1.0.dotx

### 5.2.1 Kontrollfluss

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| FOR | FOR-Schleifen mit nicht konstanter Anzahl an Schleifendurchläufe pro Zyklus, innerhalb eines Zustandes, sind nicht erlaubt | X | X | |
| WHILE / REPEAT | WHILE- und REPEAT-Schleifen sind nicht erlaubt. | X | X | |
| CONTINUE / EXIT | Die Schlüsselwörter CONTINUE und EXIT, welche die Abarbeitung der Schleife beeinflussen, sind nicht erlaubt | X | X | |

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| Return | Bedingte / Multiple Return Statements sind nicht erlaubt | X | X | |

### 5.2.2 Ausdrücke

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Komplexität | Ausdrücke mit einer maximalen Schachtelungstiefe größer 1 sind nicht erlaubt<br>Bsp.:<br>• Tiefe 1: (a + 1)<br>• Tiefe 1: (a + 1 + b)<br>• Tiefe 2: ((a*3) + b)<br>• Tiefe 3: (((a / 3) = 5) AND ((b*2) > 10)) | X | X | |
| | Ausdrücke mit mehr als 5 Operatoren oder Operanden sind nicht erlaubt | X | X | |
| Zwischen-Ergebnisse | Komplexe Ausdrücke, mit einer Schachtelungstiefe > 1, dürfen sich nicht auf implizite Casts verlassen.<br>Zwischenergebnisse werden aus Effizienzgründen, sofern möglich, auf Registerbreite gehalten. | | | X |
| Klammern | Ausdrücke, welche sich auf Vorrangregeln der Operatoren verlassen, sind nicht erlaubt | X | X | |
| Konstanten | Ungetypte Konstanten in Ausdrücken sind nicht erlaubt<br>Bsp.:<br>• nicht erlaubt: a := b + 5<br>• erlaubt: a := b + UINT#5 | X | X | |
| Zuweisungen | Mehr als eine Zuweisung pro Ausdruck ist nicht erlaubt | X | X | |

### 5.2.3 Operatoren

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| AND | Berechnung mit mehr als einem SAFEBOOL Ausgang ist nicht erlaubt | X | X | X |
| | Berechnung eines SAFEBOOL Ausgangs ohne mindestens einen SAFEBOOL Eingang ist nicht erlaubt | X | X | X |
| OR | Berechnung mit mehr als einem SAFEBOOL Ausgang ist nicht erlaubt | X | X | X |
| | Berechnung eines SAFEBOOL Ausgangs mit BOOL Eingängen ist nicht erlaubt | X | X | X |
| XOR / NOT | --- | | | |
| +, -, *, / | --- | | | |
| MIN, MAX | --- | | | |
| =, <>,<br><, <=, | --- | | | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| >, >= | | | | |

### 5.2.4 Mathematische Funktionen

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| SEL, MUX, LIMIT | --- | | | |
| MOD, EXPT, COS, ACOS, SIN, ASIN, TAN, ATAN, LOG, SQRT | Verwendung ist nicht erlaubt, sofern sie nicht explizit vom Steuerungshersteller erlaubt wurde | X | X | X |
| SHL, SHR, ROR, ROL | Shift Operationen, die größer als die Datentypbreite sind, sind nicht erlaubt. | X | X | X |

## 5.3 Funktionsplan (FUP / FBD) und Kontaktplan (KOP / LD)

### 5.3.1 Kontrollfluss

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Jump | Sprünge sind nicht erlaubt | X | | |
| | Rückwärtssprünge sind nicht erlaubt | | X | |
| Return | Bedingte / Multiple Return Statements sind nicht erlaubt | X | | |
| Verzweigung | Verzweigungen sind nicht erlaubt | X | | |

### 5.3.2 Operatoren

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| AND | Berechnung mit mehr als einem SAFEBOOL Ausgang ist nicht erlaubt | X | X | X |
| | Berechnung eines SAFEBOOL Ausgangs ohne mindestens einen SAFEBOOL Eingang ist nicht erlaubt | X | X | X |
| OR | Berechnung mit mehr als einem SAFEBOOL Ausgang ist nicht erlaubt | X | X | X |
| | Berechnung eines SAFEBOOL Ausgangs mit BOOL Eingängen ist nicht erlaubt | X | X | X |
| XOR / NOT | Verwendung ist nicht erlaubt | X | | |
| ADD, MUL, SUB, DIV | Verwendung ist nicht erlaubt | X | | |
| MIN, MAX | Verwendung ist nicht erlaubt | X | | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

### 5.3.3 Mathematische Funktionen

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| EQ, NE, LT, LE, GT, GE | Verwendung ist nicht erlaubt | X | | |
| SEL, MUX, LIMIT | Verwendung ist nicht erlaubt | X | | |
| MOVE | Verwendung ist nicht erlaubt | X | | |
| MOD, EXPT, COS, ACOS, SIN, ASIN, TAN, ATAN, LOG, SQRT | Verwendung ist nicht erlaubt, sofern sie nicht explizit vom Steuerungshersteller erlaubt wurde | X | X | X |
| SHL, SHR, ROR, ROL | Verwendung ist nicht erlaubt | X | X | |

### 5.3.4 Standard Funktionsblöcke

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| TON, TOF, TP | --- | | | |
| CTU, CTD, CTUD | --- | | | |
| Bistabile FBs (SR, RS) | Verwendung ist nicht erlaubt | X | | |
| Flanken-triggerung | Verwendung ist nicht erlaubt | X | | |

## 5.4 Kontaktplan (KOP / LD)

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Leitungen | --- | | | |
| Funktions-bausteine | Aufrufe von Funktionsbausteinen sind nicht erlaubt | X | | |
| Kontakte | Standardmäßig geschlossene Kontakte sind nicht erlaubt | X | | |
| Spulen | Negierte Spulen sind nicht erlaubt | X | | |

## 5.5 Funktionsplan (FUP / FBD)

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| Eingänge / Ausgänge | Negierte Ein- oder Ausgänge sind nicht erlaubt | X | | |

### 5.6 Continuous Flow Chart (CFC)

Die Sprache CFC ist nicht Bestandteil des IEC61131-3 Standards und wird somit weder von der PLCopen Safety Spezifikation [TC5] noch der Maschinenrichtlinie EN13849 klassifiziert. Es handelt sich dabei um eine datenflussorientierte Sprache, welche eine freie Platzierung der Blöcke erlaubt.

Durch die datenflussorientierte Verknüpfung eignet sie sich gut zur Lösung einer bestimmten Klasse von Aufgaben. Die freie Platzierung der Blöcke birgt jedoch auch ein erhöhtes Fehlerpotential beim Review solcher Diagramme.

**§H2-5.8 (CFC: Modultest erforderlich) [S3-4.10]**

Der Applikateur darf den CFC nur verwenden, wenn er einen vollständigen Modultest damit durchführt.

Aufgrund der erhöhten Komplexität der Sprache im Vergleich zu FUP / FBD oder KOP / LD, kann sie aus Sicht der PLCopen Safety auch nicht im Basic Level verwendet werden. Eine POU, welche mit CFC entwickelt wurde, muss immer vollständig Modulgetestet werden. Daher wird die Sprache auch erst ab dem Extended Level und nicht im Basic Level erlaubt.

#### 5.6.1 Allgemein

**§H2-5.9 (CFC: Platierung von Inputs und Outputs) [S3-4.11]**

Inputs sowie Outputs dürfen nur auf den Rändern des Seitenorientierten CFC platziert werden. Inputs oder Outputs inmitten des Diagramms sind nicht zulässig.

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Verwendung | Die Programmierung von POUs in CFC ist nicht erlaubt | X | | |
| Inputs / Outputs | Inputs und Outputs innerhalb des Diagramms, außerhalb des Randbereichs sind nicht erlaubt | | X | |

#### 5.6.2 Kontrollfluss

**§H2-5.10 (CFC: Sprünge sind nicht erlaubt) [S3-4.12]**

Der Applikateur darf den CFC nur verwenden, wenn er einen vollständigen Modultest damit durchführt.

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Jump | Sprünge sind nicht erlaubt | | X | |
| Return | Return Statements sind nicht erlaubt | | X | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

### 5.6.3 Operatoren

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| AND | Berechnung mit mehr als einem SAFEBOOL Ausgang ist nicht erlaubt | | X | X |
| | Berechnung eines SAFEBOOL Ausgangs ohne mindestens einen SAFEBOOL Eingang ist nicht erlaubt | | X | X |
| OR | Berechnung mit mehr als einem SAFEBOOL Ausgang ist nicht erlaubt | | X | X |
| | Berechnung eines SAFEBOOL Ausgangs mit BOOL Eingängen ist nicht erlaubt | | X | X |
| XOR / NOT | --- | | | |
| ADD, MUL, SUB, DIV | --- | | | |
| MIN, MAX | --- | | | |

### 5.6.4 Mathematische Funktionen

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| EQ, NE, LT, LE, GT, GE | --- | | | |
| SEL, MUX, LIMIT | --- | | | |
| MOVE | --- | | | |
| MOD, EXPT, COS, ACOS, SIN, ASIN, TAN, ATAN, LOG, SQRT | Verwendung ist nicht erlaubt, sofern sie nicht explizit vom Steuerungshersteller erlaubt wurde | | X | X |
| SHL, SHR, ROR, ROL | Verwendung ist nicht erlaubt | | X | |

### 5.6.5 Standard Funktionsblöcke

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| TON, TOF, TP | --- | | | |
| CTU, CTD, CTUD | --- | | | |
| Bistabile FBs (SR, RS) | --- | | | |
| Flanken-triggerung | --- | | | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

## 5.7 UML Statechart (UML/SC)

Die Sprache UML/SC ist nicht Bestandteil des IEC61131-3 Standards. Sie wurde von der Object Management Group (OMG) in dem IEC19505 Standard spezifiziert. Somit wird sie weder von der PLCopen Safety Spezifikation [TC5] noch der Maschinenrichtlinie EN13849 klassifiziert. Es handelt sich dabei um eine zustandsorientierte Sprache.

Da die Diagrammart UML Statechart zunächst Sprach- und Zielsystemunabhängig spezifiziert ist, definiert sie keine klare Zyklussynchrone Abarbeitung, sondern lediglich Events. Dies hat zur Folge, dass die Sprache nicht klar definiert wie oft eine Do-Action ausgeführt oder in welchem Zyklus eine Transition geprüft wird.

Das Verhalten im UML Statechart von CODESYS ist wie folgt:

- Do Actions werden immer innerhalb eines Zyklusses ausgeführt, in welchem der Zustand aktiv war.
- Trigger-Transitionen werden immer in einem Zyklus geprüft und sind erst beim Folgezyklus aktiv.
- Triggerless-Transitionen sind bereits im gleichen Zyklus aktiv in dem die Bedingung Wahr wird.
- Wird ein Endzustand erreicht, beginnt die Ausführung wieder von vorne.

**Beispiel 1:**

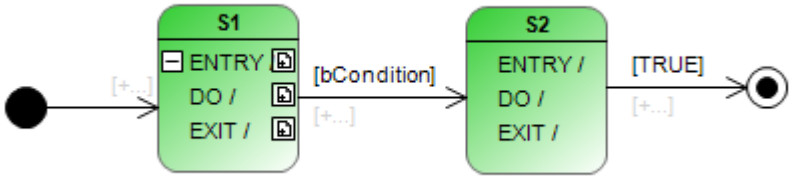
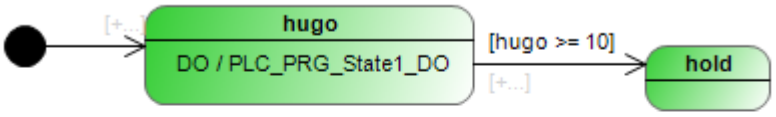
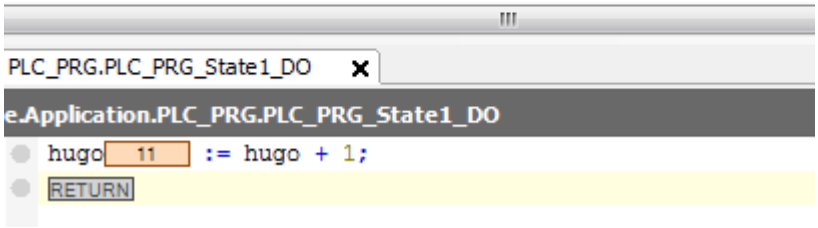



**Abbildung 3** Beispiel eines einfachen Zustandsautomates

| Zyklus | Zustandsänderung | Ausgeführte Aktionen | Anmerkung |
|---|---|---|---|
| 1 | Eintritt in Zustand "S1" | S1: Entry<br>S1: Do | |
| 2 | | S1: Do | |
| 3 | Trigger der Austrittstransition von S1 | S1: Do | Der Trigger ist erfolgt, allerdings findet der Übergang erst im nächsten Zyklus statt |
| 4 | | S1: Do<br>S1: Exit<br>S2: Entry<br>S2: Do | In diesem Zyklus wird der Übergang durchgeführt. Da sich die Zustandsmaschine innerhalb dieses Zyklusses in S1 sowie S2 befindet, werden auch beide Do-Actions aufgerufen. |

**Beispiel 2:**




Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

**Abbildung 4** Beispiel eines Zustandsautomates mit Trigger-Transition

**Beispiel 3:**

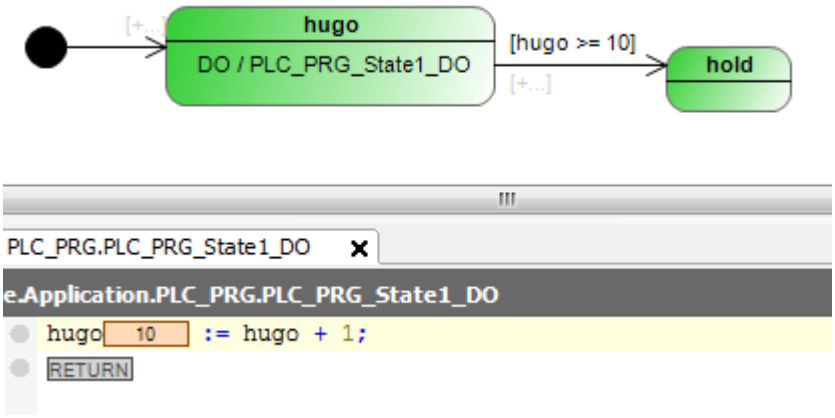



**Abbildung 5** Beispiel eines Zustandsautomates mit Triggerless-Transition

Wie in Beispiel 2 und Beispiel 3 zu sehen, ist das genaue Verhalten einer Statemachine nicht rein aus dem Diagram ablesbar, da der Unterschied zwischen einer Trigger-Transition und einer Triggerless-Transition nicht direkt ersichtlich ist. Außerdem wird an diesen Beispielen deutlich, welche Auswirkung Trigger-Transitions auf die Ausführung haben. Obwohl die Übergangsbedingung ungeachtet der Transitionsart beide Male im gleichen Zyklus aktiv war, verweilte der Statechart mit der Trigger-Transition einen vollen Zyklus länger in dem State.

Aus diesem Grund muss das korrekte Verhalten eines Statecharts immer mittels Tests nachgewiesen werden.

**§H2-5.11 (UML/SC: Test des Statechart) [S3-4.13]**

Bei der Abnahme / Zertifizierung eines UML Statechart müssen alle Zustandsübergänge getestet werden.

Ein Statechart bietet Eingänge für ReInit und Abort. Diese sind intern und dürfen daher nicht aus der Applikation heraus verwendet werden.

**§H2-5.12 (UML/SC: Verwendung der Eingänge ReInit / Abort) [S3-4.14]**

Die Eingänge ReInit und Abort des UML Statechart dürfen nicht auf Applikationsebene verwendet werden.

### 5.7.1 Statechart Elemente

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| State | States sind nicht erlaubt | X | | |
| Transition | Transitions sind nicht erlaubt | X | | |
| Exception Transition | Exception Transitions sind nicht erlaubt | X | X | |
| Completion Transition | Completion Transitions sind nicht erlaubt | X | | |
| Start State | Start States sind nicht erlaubt | X | | |
| End State | End States sind nicht erlaubt | X | | |
| Choice | Choices sind nicht erlaubt | X | X | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | | |
|---|---|---|---|---|
| Composite | Composite States sind nicht erlaubt | X | | |
| Fork / Join | Fork / Join sind nicht erlaubt | X | X | |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

# 6 Signalverarbeitung

Dieses Kapitel beschreibt Programmierregeln die eingehalten werden müssen, um sichere,- sowie unsichere Daten zu verarbeiten. Beschrieben wird der Datenfluss einer Applikation, wenn diese nach den Richtlinien aus [TC5] mit den entsprechenden Klassifizierungen zertifiziert wird.

## 6.1 Entstehung und Verarbeitung sicherer Daten

Als sichere Daten bezeichnet man solche, die vom Datentyp her als SAFE-Datentyp deklariert wurden (z.B. SAFEBOOL statt BOOL). Diese Daten können auf zwei Wegen entstehen:

1. Ein Gerät im Gerätebaum bietet bereits einen SAFE-Datentyp als Eingang an.
2. Ein Safety-Protokoll liest mehrere unsichere Daten und generiert durch deren Verifikation sichere Daten.

**§H2-6.1 (Berechnung sicherer Ausgänge) [S2-4.3]**

Sichere Ausgänge dürfen nur aufgrund von sicheren Eingangsdaten berechnet werden.

Werden die Daten unsicher von dem Gerät gelesen und durch eine Verifikation in sichere Daten gewandelt, so muss an einer bestimmten Stelle im Code eine Konvertierung von einem unsicheren Datentyp auf einen SAFE-Datentyp erfolgen. Dies kann z.B. mit einer expliziten Typkonvertierung im Extended- oder System Level geschehen.

**§H2-6.2 (I/O Geräte ohne SAFE-Datentypen) [S3-4.8]**

Stellt das verwendete Gerät, die Ein- oder Ausgänge zwar nicht als SAFE-Datentyp bereit, definiert die Ein- oder Ausgänge jedoch trotzdem als sicher, dürfen die Daten von einem - oder auf einen sicheren Datentyp, gecastet werden. Hierzu ist über einen Review dieser expliziten Casts sicherzustellen, dass die Umwandlung der tatsächlichen Belegung entspricht.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

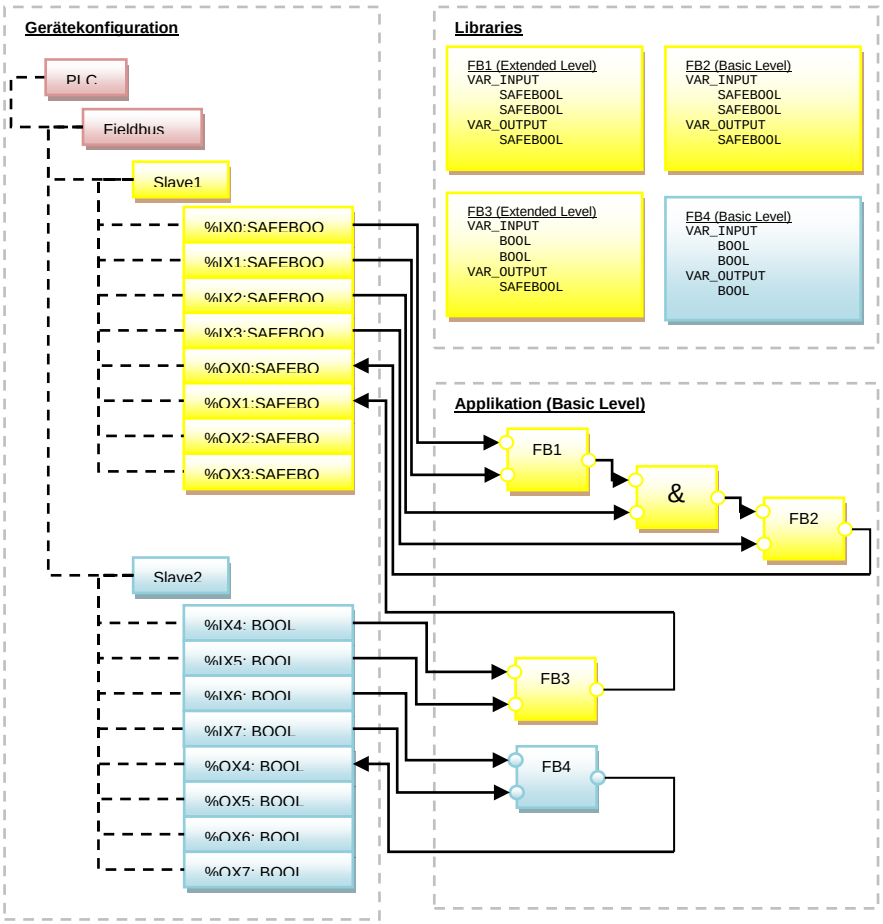



**Abbildung 6** Beispiel für den Datenfluss einer sicheren Applikation

## 6.2 Signalpegel

### §H2-6.3 (Defaultwerte und Ruhestromprinzip) [S3-4.9]

Die Signalpegel in einem sicherheitsgerichteten System basieren nach [TC5] 3.3 auf dem "Ruhestromprinzip". Dies bedeutet in Bezug auf die Software, dass der Wert FALSE immer den sicheren Zustand definieren muss.

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Pegel | Ein Default Wert ungleich 0 (bzw. FALSE) für Aktivierungseingänge ist nicht erlaubt | X | X | X |
| | Bei einem Wert von 0 an allen Eingängen darf sich der Funktionsbaustein oder das Programm nicht im Betriebsmodus befinden | X | X | X |
| Ausgänge | Ausgänge, die während eines Zyklusses nicht neu beschrieben werden, sind nicht erlaubt | X | X | |

## 6.3 Verarbeitung nicht sicherer Daten

CODESYS Safety SIL2 bietet die Möglichkeit rückwirkungsfrei Code zu verwenden, welcher nicht nach den anzuwendenden Sicherheitsstandards entwickelt wurde. Hierzu bietet CODESYS Safety SIL2 im wesentlichen

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

zwei Konzepte. Diese beiden Konzepte sind optional und stehen nicht auf jeder Steuerung zur Verfügung. Konsultieren Sie also im Zweifel die Dokumentation des Steuerungsherstellers.

### 6.3.1 Compound Safety SIL2

Unterstützt eine Steuerung das Compound Safety Feature, können zwei unabhängige Applikationen auf die Steuerung geladen werden. Eine dieser Applikationen ist die Safety Applikation, die andere ist die Standard Applikation.

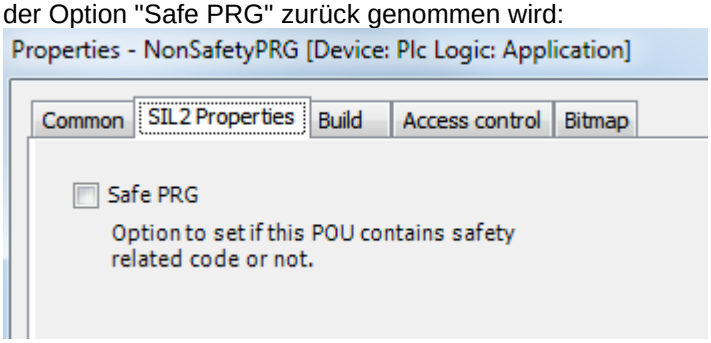
Dieses Feature hat den Vorteil, dass an der Standard Applikation Änderungen vorgenommen werden können ohne die Safety Applikation neu abnehmen zu müssen.

Nähere Informationen zur Verwendung finden Sie in der Online Hilfe von CODESYS oder in der Dokumentation des Steuerungsherstellers.

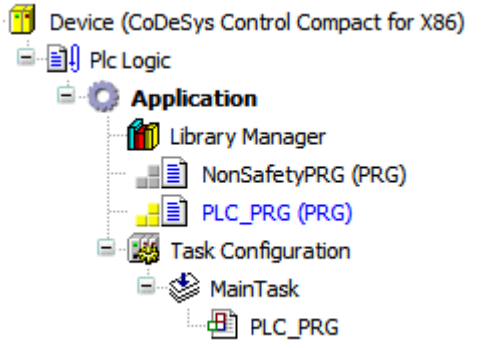
### 6.3.2 Non-Safe PRGs

Unterstützt eine Steuerung sog. Non-Safe PRGs, kann eine Applikation einzelne Programme enthalten, welche nicht zertifizierten Code enthalten. Dieser Code wird dann rückwirkungsfrei ausgeführt und bekommt keine Schreibrechte auf Daten der sicheren GVLs, FBs oder Programme.

1. Im Property Dialog der PRG kann ein Programm als "Non-Safe" markiert werden, indem der Haken bei der Option "Safe PRG" zurück genommen wird:




2. Fortan wird die PRG mit einem grauen Symbol markiert:




Um zwischen den sicheren und den unsicheren Programmteilen Daten auszutauschen gibt es im wesentlichen zwei Möglichkeiten:

1. Sie verwenden Variablen aus VAR_INPUT und VAR_OUTPUT der Non-Safe PRG zum Datenaustausch
2. Sie verwenden eine spezielle Art der GVL, welche "EVL" (=Exchange Variable List) genannt wird.

In beiden Fällen liegen die Daten in einem ungeschützten Bereich und können somit von den sicheren wie den unsicheren Applikationsteilen verwendet werden.

**Hinweis:** Wollen Sie nicht sichere Ausgänge aus Ihrer Non-Safe PRG heraus beschreiben, müssen Sie diese auf bestehende Variablen innerhalb des VAR_OUTPUT Bereiches der Non-Safe PRG oder der EVL mappen.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

### 6.4 FB Schnittstellen

[TC5] definiert eine Standardschnittstelle, die jeder sichere FB erfüllen muss. Die folgenden Parameter sind dort beschrieben:

**Eingänge:**

- **Activate: BOOL**
  Typ: Variable oder Konstante
  Dieser Eingang aktiviert den Funktionsbaustein. Der Initialwert ist FALSE. Dieser Parameter kann mit einer Variablen Verbunden werden, die den aktuellen Status (Aktiv/Inaktiv) des Safety Gerätes angibt. Wenn die Variable TRUE ist, ist der FB aktiv - wenn sie FALSE ist, müssen die Ausgänge auf den Default Wert gesetzt werden.
  Wenn kein Gerät verbunden ist, muss die Variable mit einer TRUE Konstante verbunden werden.
- **S_<Sicherer Input Name>: SAFExxxx**
  Der Name eines jeden sicheren Eingangs beginnt mit einem S_.
- **S_StartReset: SAFEBOOL**
  Typ: Variable oder Konstante
  Der Initialwert ist FALSE, was bedeutet, dass der Funktionsbaustein manuell (über den Eingang "Reset") nach einem Start der Steuerung (warm oder kalt) resetted werden muss. TRUE bedeutet, dass der FB nach dem Start der Steuerung automatisch resetted wird.
  **Anm.:** [TC5] Kapitel 5.1 fordert, dass dieser Eingang nur verwendet werden darf, wenn sichergestellt ist, dass keine Gefährdung beim Start der Steuerung auftreten kann. In CODESYS Safety SIL2 wird die Startphase abgesichert und Ausgänge werden nur nach erfolgreicher Ablaufkontrolle geschrieben. Daher ist eine Gefährdung durch die Steuerung selbst ausgeschlossen, und der Eingang kann somit verwendet werden.
- **S_AutoReset: SAFEBOOL**
  Typ: Variable oder Konstante
  Der Initialwert ist FALSE, was bedeutet, dass der Funktionsbaustein manuell (über den Eingang "Reset") nach der Freigabe des Notaus-Schalters resetted werden muss. TRUE bedeutet, dass der FB nach der Freigabe des Notaus-Schalters automatisch resetted wird.
  **Anm.:** Es gilt die gleiche Richtlinie wie in S_StartReset.
- **Reset: BOOL**
  Typ: Variable
  Dieser Eingang ist Flankengesteuert und nur bei einer steigenden Flanke aktiv. Abhängig von der Funktion des FBs kann dieser Eingang für unterschiedliche Zwecke verwendet werden. Die genaue Bedeutung des Eingangs muss in der Dokumentation des FBs beschrieben sein:
  - Reset des Zustands der Maschine und der damit verbundenen Error-Messages / Diagnose Codes, wenn der Fehler beseitigt wurde.
    **=>** Fehler Reset.
  - Manueller Reset einer "Wiederanlaufsperre" durch den Bediener der Anlage.
    **=>** Funktionaler Reset
  - Zusätzliche FB-spezifische Reset Funktionen

**Ausgänge:**

- **Ready: BOOL**
  Wenn dieser Ausgang TRUE ist, signalisiert dies, dass der FB aktiv ist, und die Ausgänge gültig sind. Er erfüllt somit die gleiche Funktion wie eine "POWER" LED eines Safety Relays. Wenn der Ausgang FALSE ist, ist der FB nicht aktiv und das Programm wird nicht ausgeführt. Dies ist im Debug Mode nützlich, sowie zur sequenziellen Aktivierung/Deaktivierung mehrerer FBs.
- **S_<Sicherer Output Name>: SAFExxxx**
  Der Name eines jeden sicheren Ausgangs beginnt mit einem S_.
- **Error: BOOL**
  Wenn dieser Ausgang TRUE ist, signalisiert dies einen Fehler im FB. Er erfüllt somit die gleiche Funktion wie eine "K1/K2" LED an einem Safety Relays. Die Art des Fehlers wird über den Ausgang "DiagCode" signalisiert. Wenn der Wert TRUE ist, befindet sich der FB in einem Fehlerzustand.
  Ist der Ausgang FALSE, bedeutet dies, dass der FB keinen Fehler festgestellt hat.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

- **DiagCode: WORD**
  Alle Zustände (Aktiv, Inaktiv und Error) werden über diesen Diagnose Code nach außen signalisiert. Falls mehrere Fehler auftraten, signalisiert dieser Diagnose Code den ersten aufgetretenen Fehler. Das Format der Diagnosecodes ist in [TC5] Kapitel 5.2 beschrieben.
  Der "DiagCode" muss immer im gleichen Zyklus wie der Zustandsübergang beschrieben werden, um konsistent den aktuellen Zustand des FBs wiederzugeben.

Dieses Interface muss nicht zwingend in allen Funktionsbausteinen, aller Levels angeboten werden. Die folgenden Regeln definieren, wie das Interface auf Funktionsbausteine der verschiedenen Level anzuwenden ist:

| Schlüsselwort | Regel | basic | extended | system |
|---|---|---|---|---|
| Allgemein | Parameter in sicheren FBs mit den, in Kapitel 6.3 definierten Namen, dürfen nicht von dem dort definierten Verhalten abweichen. | X | X | X |
| Activate | Ein sicherer FB ohne einen "Activate" Eingang ist nicht erlaubt | X | X | |
| Ready | Ein sicherer FB ohne einen "Ready" Ausgang ist nicht erlaubt | X | X | |
| Error | Ein sicherer FB ohne einen "Error" Ausgang ist nicht erlaubt | X | X | |
| DiagCode | Ein sicherer FB ohne einen "DiagCode" Ausgang ist nicht erlaubt | X | X | |
| Reset | Ein sicherer FB, welcher in einem Zustand ohne Timeout verweilen kann und keinen Reset Eingang besitzt, ist nicht erlaubt<br>**Ausnahme:** Bei dem Zustand handelt es sich um einen "sicheren Zustand" aus dem der FB sich nicht wieder erholen kann. | X | X | |

### 6.5 Verarbeitung unsicherer E/As

Zur Verarbeitung unsicherer Ein- und Ausgänge gibt es keine besonderen Regeln zu beachten. Allerdings ist beim Zugriff auf die Ein- und Ausgangsabbilder zu beachten, dass die Regeln aus 5.1.6 eingehalten werden.

Das heißt, wenn ein Gerät bzw. ein Feldbus, sichere und unsichere E/As anbietet, dürfen diese nur aus einer Task heraus verwendet werden. Bei der Verarbeitung der Daten ist allerdings darauf zu achten, dass keine sicheren Ausgänge aus unsicheren Eingängen berechnet werden.

Template: Vorlage_tecdoc_V1.0.dotx

### 6.6 CANopen Safety

CANopen Safety ist ein Sicherheitsprotokoll, basierend auf dem CANopen Standard. Architektonisch ist das CANopen Safety Protokoll oberhalb des CANopen Stacks angesiedelt. In CODESYS Safety SIL2 bedeutet dies als logische Konsequenz, dass CANopen Safety als ein Protokoll basierend auf dem CANopen Stack implementiert ist.

#### 6.6.1 Einsatz und Zertifizierung

Das Protokoll ist in CODESYS Safety SIL2 gemäß EN50325-5 implementiert. Zum Einsatz des CANopen Safety Protokolls ist es jedoch nötig die gesamte Konfiguration des Systems zu betrachten und die Spezifikation des Protokolls zu kennen.

Das Protokoll kennen müssen Sie allerdings nur um die Fehlerfälle innerhalb des Gesamtsystems zu beherrschen. Mögliche Fehler innerhalb des CANopen Safety Stacks werden von CODESYS Safety SIL2 bereits behandelt, sofern Sie die nachfolgenden Regeln befolgen.

© 3S-Smart Software Solutions GmbH

Diese Regeln gelten, sofern nicht anders vermerkt, immer für den CANopen Safety Master sowie den CANopen Safety Device Stack von 3S.

### 6.6.2 Konfiguration

Die gesamte Konfiguration des CAN Busses muss vor der Abnahme auf ihre Richtigkeit überprüft werden. Dabei sind insbesondere die folgenden Fehlerfälle zu überprüfen:

- Korrektheit und Überschneidungsfreiheit der PDO Mappings
- Korrektheit und Überschneidungsfreiheit der SRDO Mappings
- Keine Überschneidungen von CobIDs
- Keine Überschneidungen von NodeIDs
- Keine unterschiedlichen Baudraten der Teilnehmer

**§H2-6.4 (CANopen Safety Stack: Review der Konfiguration) [S2-4.8]**

Der Applikateur ist für die Richtigkeit der Konfiguration verantwortlich. Dabei muss er nicht nur die Konfiguration der sicheren I/Os sondern die gesamte Buskonfiguration innerhalb und außerhalb seiner CANopen Konfiguration in CODESYS betrachten.

Hierbei gilt es insbesondere die folgenden möglichen Fehlkonfigurationen zu überprüfen:

1. Korrektheit und Überschneidungsfreiheit der PDO Mappings
2. Korrektheit und Überschneidungsfreiheit der SRDO Mappings
3. Überschneidungen von CobIDs
4. Überschneidungen von NodeIDs

Unterschiedliche Baudraten der Teilnehmer

Ist auf dem CANopen Bus SRDO Kommunikation direkt zwischen zwei CANopen Safety Slaves konfiguriert, muss der Applikateur die Fehlerfälle für diese Kommunikation im White-Box Verfahren betrachten.

Um diese detaillierte Betrachtung zu umgehen, kann der CANopen Master auch die Überwachung des TxSRDOs übernehmen und somit die Reaktion auf eine Störung der Kommunikation zwischen den Teilnehmern übernehmen.

**§H2-6.5 (CANopen Safety Stack: Querkommunikation) [S2-4.15]**

Konfiguriert der Applikateur seine PDO- oder SRDO Mappings so, dass die Kommunikation gänzlich an dem Master vorbei geht, müssen auch alle Fehlerfälle die sonst von dem Master behandelt werden von dem Applikateur betrachtet werden.

Bieten die Slaves nicht die nötigen Diagnosemöglichkeiten, kann der CANopen Master auch zur Überwachung der TxSRDOs verwendet werden.

Template: Vorlage_tecdoc_V1.0.dotx

In CODESYS Safety SIL2 sind Strukturen und Arrays als Datentypen für I/O Kanäle von CANopen Safety Slaves nicht erlaubt. Sie sollten daher auch von keinem CANopen Slave angeboten werden.

Die I/O Kanäle dürfen immer nur komplett und nicht teilweise gemappt werden. Einzige Ausnahme hierbei sind Bitkanäle, welche auf bestehende Variablen gemappt werden dürfen.

© 3S-Smart Software Solutions GmbH

**§H2-6.6 (CANopen Safety Stack: Mapping von Arrays, Strukturen und partielles Mapping) [S2-4.9]**

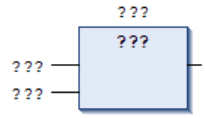
Parameter und Strukturen sind im I/O Mapping von CANopen Safety Slaves nicht erlaubt. Außerdem dürfen Parameter nicht nur partiell gemappt werden.

Ist in der I/O Konfiguration also z.B. ein DWORD definiert, darf nicht nur ein Byte daraus gemappt werden.

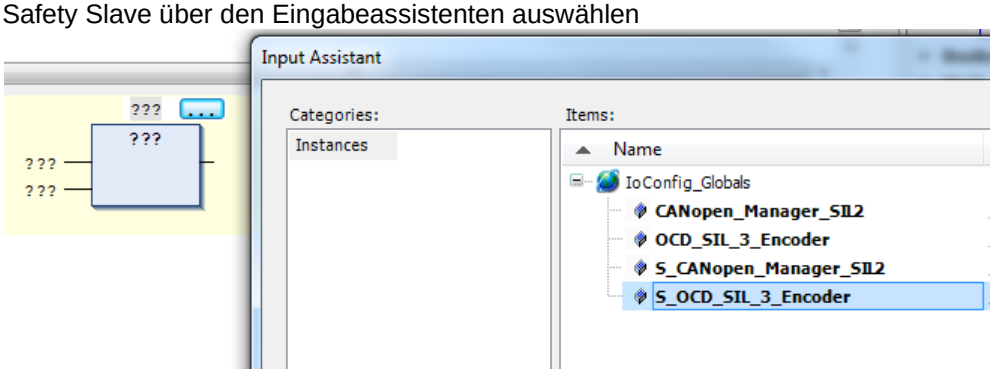
### 6.6.3 Diagnose

Sichere Eingänge, welche nicht ohnehin strikt dem Ruhestromprinzip folgen, müssen beim Lesen immer über eine Sichere Diagnose abgesichert werden. Bei der Verwendung des 3S CANopen Safety Stacks wird hierzu ein Diagnose Ausgang an jedem Slave Baustein bereit gestellt.

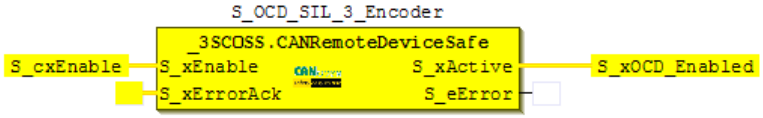
1. Leeren Baustein einfügen




2. Safety Slave über den Eingabeassistenten auswählen




3. S_xActive und ggf. S_eError auswerten




Eine ähnliche Überwachung kann ebenfalls erzielt werden, wenn der Slave ein eigenes TxSRDO zur Signalisierung des aktiven Zustandes bereitstellt.

**§H2-6.7 (CANopen Safety Stack: Auswertung der Diagnose) [S2-4.10]**

Beim Lesen sicherer Eingänge eines Slaves muss der Applikateur auch immer die sichere Diagnose des jeweiligen Slave Bausteins auswerten.

Reine Aktoren, die keine Sensorwerte liefern, bieten der Applikation selbst keine Diagnosemöglichkeit. Daher müssen diese auf einem anderen Weg (z.B. durch einen sicheren Sensor) überwacht werden.

Das Monitoring im CODESYS Safety Programmiersystem ist generell nicht sicher. Somit ist auch die Statusanzeige der Geräte im Gerätebaum nicht 100%ig zuverlässig. Der Status im Gerätebaum kann somit zur Fehleranalyse verwendet, darf aber nicht zur Risikobewertung des aktuellen Zustandes des Gesamtsystems herangezogen werden.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

**§H2-6.8 (CANopen Safety Stack: Monitoring und Diagnose im Gerätebaum ist unsicher) [S2-4.14]**

Das Monitoring in CODESYS Safety SIL2 ist generell unsicher. Daher ist auch die Statusanzeige der Geräte im Gerätebaum unsicher. Der Applikateur darf sich daher nicht 100%ig auf die Anzeige verlassen.

Nach der Betrachtung des Gesamtsystems ist es möglich, dass ein Slave im laufenden, sicheren Betrieb aufgrund, eines Fehlers in unsicheren Systemteilen, neu konfiguriert wird. Dies kann dazu führen, dass die Ausgänge eines Slaves kurzzeitig ihren Failsafe Wert annehmen, danach aber gleich wieder auf ihren ursprünglichen Wert wechseln.

Falls sich die Konfiguration zusätzlich noch von der ursprünglichen Konfiguration unterscheidet würde es im schlimmsten Fall sogar dazu führen, dass falsche Ausgänge geschrieben oder falsche Eingänge gelesen würden.

Folgende Maßnahmen können zur Beherrschung dieser Fehlerfälle herangezogen werden:

- Wenn die Ein- und Ausgänge kurzzeitig den Failsafe Wert annehmen dürfen:
  - Einsatz von statisch konfigurierten Slaves
  - Testen aller Sensoren und Aktoren auf korrekte Funktion innerhalb der Process Safety Time
- Einrichtung einer Wiederanlaufsperre an allen Slaves, sodass eine solche ungewollte Rekonfiguration in einen sicheren Zustand führt.

**§H2-6.9 (CANopen Safety Stack: Ungewollte Rekonfiguration im Betrieb) [S2-4.11]**

Der unsichere CANopen Master Stack ist potentiell in der Lage einen Slave neu zu konfigurieren. Geschieht dies im laufenden Betrieb, kann dies zu unterschiedlichen Fehlerbildern führen. Diese muss der Applikateur in seiner Fehlerbetrachtung berücksichtigen.

Die folgenden Maßnahmen wären zur Fehlerbeherrschung möglich:

- Wenn die Ausgänge kurzzeitig ihren Fail Safe Wert annehmen dürfen:
  a. Einsatz von statisch konfigurierten Slaves
  b. Testen des Sensorwertes innerhalb der Process Safety Time

Wiederanlaufsperre an den Slaves, sodass diese ungewollte Rekonfiguration zu einem Fehlerzustand führt.

Template: Vorlage_tecdoc_V1.0.dotx

### 6.6.4 Limitierungen

#### 6.6.4.1 Lizenzierung und Demo Modus

Der Standard CANopen Stack kann testweise auch ohne gültige Lizenz im sog. "Demo Modus" betrieben werden. In diesem Modus ist der Stack voll funktionsfähig, die Laufzeit ist jedoch begrenzt. Diese Funktionalität ist bei der Verwendung des CANopen Safety Stacks deaktiviert um zu vermeiden, dass durch eine korrumpierte Lizenz die Maschine zwar startet, aber nach Ablauf der Demozeit die CAN Kommunikation einfach einstellt.

Eine fehlende Lizenz verhindert den Start CANopen Safety Stacks. Eine sichere Kommunikation kommt somit nicht zu Stande.

© 3S-Smart Software Solutions GmbH

**§H2-6.10 (CANopen Safety Stack: Demo Modus nicht möglich) [S2-4.12]**

Um einen ungeplanten Ausfall der Maschine aufgrund eines fälschlicherweise gesetzten Demo-Moduses zu verhindern, ist der Demo-Modus für den CANopen Stack unter Verwendung von CANopen Safety abgeschaltet.

Haben der CANopen Stack oder CANopen Safety keine Lizenz, laufen beide Busse gar nicht erst an.

#### 6.6.4.2 Global Failsafe Command

Das Global Failsafe Command (kurz GFC) ist in EN50325-5 als ein Telegramm spezifiziert, welches von den Teilnehmern gesendet werden kann um einen Fehlerzustand zu signalisieren. Da dieses Telegramm allerdings weder sicher übertragen wird, noch eine nähere Information über die Fehlerursache mitgeführt wird, wird dieses Kommando vom 3S CANopen Safety Stack nicht zur sicheren Diagnose unterstützt.

**§H2-6.11 (CANopen Safety Stack: GFCs sind nicht sicher) [S2-4.13]**

Im CANopen Safety Protokoll werden GFCs auf nicht sicherem Wege übertragen. Daher darf der Applikateur sich in seiner Sicherheitsbetrachtung nicht auf ein gesendetes GFC im Fehlerfall verlassen.

### 6.7 FSoEBase

Die Bibliothek FSoEBase implementiert vollständig das FSoE Protokoll. Dabei sind allgemeine Teile, sowie spezifische Teile für die Master- und die Slave Seite enthalten. Eingehende FSoE-PDUs können verarbeitet werden und entsprechend ausgehende FSoE-PDUs erstellt werden. Alle sicherheitsspezifischen Funktionen und Prüfungen des FSoE Protokolls sind enthalten und die Ausführung erfolgt aus dem sicheren Kontext.

Die Umsetzung erfolgt als externe Bibliothek, d.h. der FB als auch die Methoden des FBs werden als Komponente des CODESYSControlSIL2 Runtimes implementiert. Beim Laden der IEC-Applikation werden die extern implementierten Methoden mit den IEC-Aufrufen verlinkt.

Das Transportmedium für FSoE-PDUs stellt keine Sicherheitsanforderungen und kann frei gewählt werden. Die Übertragung der FSoE-PDUs erfolgt somit aus dem nicht sicheren Kontext. Die Übertragung ist nicht Teil der Bibliothek und muss vom Verwender dieser Bibliothek implementiert werden.

Die FSoEBase Bibliothek ist somit die Grundlage für weitere Bibliotheken, welche die Implementierung des FSoE-Protokolls als Übertragungsprotokoll benötigen. Eine solche Bibliothek wäre z.B. die sichere Querkommunikation zwischen CODESYS Applikationen oder eine CODESYS IO Treiber zum Einbinden von sicheren FSoE Geräten.

Template: Vorlage_tecdoc_V1.0.dotx

#### 6.7.1 Einsatz und Zertifizierung

Das Protokoll ist in CODESYS Safety SIL2 gemäß der ETG-Spezifikation ETG5100. Implementiert. Zum Einsatz des FSoE Protokolls ist es notwendig die gesamte Konfiguration des Systems zu betrachten und die Spezifikation des Protokolls zu kennen.

Das Protokoll muss allerdings nur bekannt sein, um die Fehlerfälle innerhalb des Gesamtsystems zu beherrschen. Mögliche Fehler innerhalb der FSoE Implementierung in der FSoEBase Bibliothek werden bereits von CODESYS Safety SIL2 behandelt.

Weiterhin muss vor der Verwendung der FSoEBase Bibliothek der FSoE Conformance Test auf der Zielplattform durchgeführt worden sein. Dieser kann bereits vom Gerätehersteller durchgeführt worden sein.

**§H2-6.15 (FSoEBase: Conformance Test) [S2-4.24]**

Der FSoE Conformance Test muss auf dem Zielgerät durchgeführt worden sein.

© 3S-Smart Software Solutions GmbH

Die nachfolgenden Regeln müssen bei der Verwendung der FSoEBase Bibliothek befolgt werden. Diese Regeln gelten, sofern nicht anders vermerkt, immer sowohl für den FSoE Master, als auch für den FSoE Slave.

### 6.7.2 Konfiguration

Die gesamte FSoE Konfiguration muss vor der Abnahme auf ihre Richtigkeit überprüft werden. Dabei sind insbesondere die folgenden Fehlerfälle zu überprüfen:

- Korrektheit und Überschneidungsfreiheit der FSoE Adressen im erreichbaren Netzwerk
- Korrektheit und Überschneidungsfreiheit der FSoE ConnectionIDs im erreichbaren Netzwerk
- Der Inhalt und das Layout der Applikationsparameter, welche vom Master an den Slave übertragen werden, stimmen überein
- Der Inhalt und das Layout der Kommunikationsparameter zwischen Master und Slave stimmen überein
- Das Layout der Nutzdaten zwischen Master und Slave stimmen überein.

**§H2-6.12 (FSoEBase: Review der Konfiguration) [S2-4.19]**

Der Applikateur ist für die Richtigkeit der Konfiguration verantwortlich.

Hierbei gilt es insbesondere die folgenden möglichen Fehlkonfigurationen zu überprüfen:

1. Korrektheit und Überschneidungsfreiheit der FSoE Adressen im erreichbaren Netzwerk
2. Korrektheit und Überschneidungsfreiheit der FSoE ConnectionIDs im erreichbaren Netzwerk
3. Inhalt und Layout der Applikationsparameter zwischen Master und Slave stimmen überein
4. Inhalt und Layout der Kommunikationsparameter zwischen Master und Slave stimmen überein
5. Das Layout derr Nutzdaten zwischen Master und Slave stimmen überein

Die Applikationsparameter müssen vom Slave unmittelbar nach Erhalt der Kommunikations – und Applikationsparameter geprüft werden. Dies muss vor der ersten Verwendung der Nutzdaten erfolgt sein, da die Übertragung sonst unter falschen Bedingungen stattfindet.

Bei der Verwendung der FSoEBase Bibliothek aus der Applikation, muss darauf geachtet werden, in welchem Zyklus die Funktionsbausteine aufgerufen werden. Die Implementierung überprüft die Watchdog-Zeit in der Methode Input anhand der aktuellen Zeit. Deshalb muss diese Methode mindestens doppelt so oft wie die eingestellte Watchdog Zeit aufgerufen werden.

**§H2-6.13 (FSoEBase: Aufrufe der Bibliothek) [S2-4.22]**

Da der FsoE Stack sich auf die Watchdog Diagnose verlässt, muss sichergestellt werden, dass die Input Methode der Bibliothek mindestens doppelt so oft wie die eingestellte Watchdog Zeit aufgerufen wird.

Template: Vorlage_tecdoc_V1.0.dotx

### 6.7.3 Limitierungen

#### 6.7.3.1 Lizensierung und Demo Modus

Gewöhnlich werden die Feldbusstacks ohne gültige Lizenz im sog. „Demo Modus“ betrieben. In diesem Modus ist der Stack voll funktionsfähig, die Laufzeit ist jedoch begrenz.

© 3S-Smart Software Solutions GmbH

Diese Funktionalität ist beim FSoEBase Stack deaktiviert um zu vermeiden, dass durch eine korrumpierte Lizenz die Maschine zwar startet, aber nach Ablauf der Demozeit die Kommunikation einfach einstellt.

Eine fehlende Lizenz verhindert den Start des FSoEBase Stacks. Eine sichere Kommunikation kommt somit nicht zu Stande.

**§H2-6.14 (FSoEBase: Demomodus nicht möglich) [S2-4.21]**

Um einen ungeplanten Ausfall der Maschine aufgrund eines fälschlicherweise gesetzten Demo-Moduses zu verhindern, ist der Demo-Modus für die FSoEBase Bibliothek abgeschaltet.

Hat die Bibliothek FSoEBase keine Lizenz, läuft die Kommunikation gar nicht erst an.

### 6.7.3.2 Wiederanlauf und Fehleraufdeckung

Zur Einhaltung der Fehlerraten durch das implementierte Protokoll sind folgende Voraussetzungen zu erfüllen:

**§H2-6.16 (FSoEBase: Voraussetzungen für Fehleraufdeckung) [S2-4.25]**

Ein FSoE Telegramm muss folgende Voraussetzungen erfüllen:

- die Anzahl der Safety Datenbits in einem Telegramm ist entweder 8 oder 16
- die maximale Anzahl der virtuellen Bits in einem Telegramm ist 24
- die minimale Anzahl der Standard Bits in einem Telegramm ist 16
- die maximale Anzahl der Standard Bits in einem Telegramm ist 12144

Weiterhin muss für den dauerhaften Betrieb folgende Anforderung erfüllt werden um Fehlerhäufung zu vermeiden:

**§H2-6.17 (FSoEBase: Wiederanlauf) [S2-4.23]**

Ein Reset des Systems nach einem erkannten Fehler und dem Erreichen des sicheren Zustands darf nur einmal in 5 Stunden Betriebszeit durchgeführt werden.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

# 7 Anhang

## 7.1 Abweichungen von PLCopen Safety

In dem Dokument [TC5] des PLCopen Safety Komitees ist ein Programmierstandard beschrieben, welcher für Software, die mit einer LVL geschrieben wird, und nach EN62061 zertifiziert werden soll, ausreicht. In dem Dokument [TC5] wird nicht auf einen bestimmten SIL eingegangen. Jedoch liegt der Hauptfokus eindeutig auf SIL3 ohne Mischung von funktional sicherem und –unsicherem Code. Daher ergeben sich für eine SIL2 Umgebung, welche außerdem sicheren- sowie unsicheren Code in der Applikation zulässt, leichte Änderungen.

1. [TC5] Kapitel 3.3
   Hier wird zwischen „Safety-„ und „Functional Application“ unterschieden, die auf unterschiedlichen Prozessoren laufen. Dies ist ein Szenario aus gängigen SIL3 Systemen.
   Unter CODESYS Safety SIL2 sind mehrere Tasks erlaubt und es wird nicht strikt zwischen sicherheitsgerichteter und funktionaler Applikation unterschieden.
2. [TC5] Kapitel 3.3
   Sichere Applikationsteile dürfen unter CODESYS Safety SIL2 von unsicheren unterbrochen werden, sofern diese rückwirkungsfrei sind.
3. [TC5] Kapitel 4.3
   WORD ist erlaubt.
4. [TC5] Kapitel 4.3
   REAL Variablen im Basic oder Extended Level müssen explizit vom Steuerungshersteller erlaubt werden.
5. [TC5] Kapitel 4.4
   Die Beschränkung für OR im Basic Level wurde aufgelockert. Es können somit in einer Applikation, welche im Basic Level geschrieben ist, auch unsichere I/Os verschaltet werden.
6. [TC5] Kapitel 4.4
   MOVE ist nicht kategorisch verboten, sondern nur im Basic Level nicht erlaubt.
7. [TC5] Kapitel 4.4
   MOD und EXPT werden zusammen mit den Arithmetischen Funktionen behandelt.
8. [TC5] Kapitel 4.4
   In [TC5] ist nur die Rede von „Unary REAL functions“. In dem hier vorliegenden Dokument sind alle Funktionen einzeln gelistet (wozu auch MOD und EXPT zählen) und zunächst in allen Kategorien verboten. Sie können allerdings vom Steuerungshersteller explizit erlaubt werden.

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

## 7.2 Abbildungsverzeichnis

**Abbildung 1** Normenübersicht ....9
**Abbildung 2** Programmstruktur ....11
**Abbildung 3** Beispiel eines einfachen Zustandsautomates ....24
**Abbildung 4** Beispiel eines Zustandsautomates mit Trigger-Transition ....25
**Abbildung 5** Beispiel eines Zustandsautomates mit Triggerless-Transition ....25
**Abbildung 6** Beispiel für den Datenfluss einer sicheren Applikation ....28

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

## 7.3 Tabellenverzeichnis

**Tabelle 1** Entsprechung von PL zu SIL.....7

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

## 7.4 Sicherheitsanforderungen

**§H2-1.1 (Einhaltung der Sicherheitsanforderungen) [S3-4.2] ..... 4**
**§H2-2.1 (Unsicheres Monitoring) [S2-4.7] ..... 6**
**§H2-2.2 (Sicheres Logging) [S2-4.18] ..... 7**
**§H2-3.1 (Abnahme der IEC Applikation) [S2-4.2] ..... 8**
**§H2-3.2 (Review der IEC Applikation) [S2-4.17] ..... 8**
**§H2-3.3 (Archivierung der IEC Applikation) [S2-4.16] ..... 9**
**§H2-4.1 (Doppelte Ausführung) [S2-4.4] ..... 10**
**§H2-4.2 (Doppeltes I/O Abbild) [S2-4.4] ..... 11**
**§H2-4.3 (Source Code Reviews) [S3-4.3] ..... 11**
**§H2-4.4 (Warnungen im Code) [S2-5.1] ..... 11**
**§H2-4.5 (Verwendung von Bibliotheken) [S3-4.4], [S2-4.1] ..... 12**
**§H2-5.1 (Erlaubte Programmiersprachen) [S3-4.5] ..... 12**
**§H2-5.2 (Gleitkomma Operationen) [S3-4.1] ..... 14**
**§H2-5.13 (Nicht natürlich alignte Strukturen) [S3-4.16] ..... 14**
**§H2-5.3 (Multitasking und I/Os) [S2-4.5] ..... 15**
**§H2-5.4 (Mehrfachmappings von I/Os) [S2-4.6] ..... 15**
**§H2-5.5 (Erlaubte Bibliotheken) [S3-4.4], [S2-4.1] ..... 16**
**§H2-5.6 (Validierung von Bibliotheken) [S3-4.6] ..... 16**
**§H2-5.14 (Verwendung der Operatoren AND_THEN / OR_ELSE) [S3-4.17] ..... 17**
**§H2-5.15 (Verwendung von Pragmas) [S3-4.15] ..... 17**
**§H2-5.7 (Strukturierter Text als LVL) [S3-4.7] ..... 18**
**§H2-5.8 (CFC: Modultest erforderlich) [S3-4.10] ..... 22**
**§H2-5.9 (CFC: Platierung von Inputs und Outputs) [S3-4.11] ..... 22**
**§H2-5.10 (CFC: Sprünge sind nicht erlaubt) [S3-4.12] ..... 22**
**§H2-5.11 (UML/SC: Test des Statechart) [S3-4.13] ..... 25**
**§H2-5.12 (UML/SC: Verwendung der Eingänge ReInit / Abort) [S3-4.14] ..... 25**
**§H2-6.1 (Berechnung sicherer Ausgänge) [S2-4.3] ..... 27**
**§H2-6.2 (I/O Geräte ohne SAFE-Datentypen) [S3-4.8] ..... 27**
**§H2-6.3 (Defaultwerte und Ruhestromprinzip) [S3-4.9] ..... 28**
**§H2-6.4 (CANopen Safety Stack: Review der Konfiguration) [S2-4.8] ..... 32**
**§H2-6.5 (CANopen Safety Stack: Querkommunikation) [S2-4.15] ..... 32**
**§H2-6.6 (CANopen Safety Stack: Mapping von Arrays, Strukturen und partielles Mapping) [S2-4.9] ..... 33**
**§H2-6.7 (CANopen Safety Stack: Auswertung der Diagnose) [S2-4.10] ..... 33**
**§H2-6.8 (CANopen Safety Stack: Monitoring und Diagnose im Gerätebaum ist unsicher) [S2-4.14] ..... 34**
**§H2-6.9 (CANopen Safety Stack: Ungewollte Rekonfiguration im Betrieb) [S2-4.11] ..... 34**
**§H2-6.10 (CANopen Safety Stack: Demo Modus nicht möglich) [S2-4.12] ..... 35**
**§H2-6.11 (CANopen Safety Stack: GFCs sind nicht sicher) [S2-4.13] ..... 35**
**§H2-6.15 (FSoEBase: Conformance Test) [S2-4.24] ..... 35**
**§H2-6.12 (FSoEBase: Review der Konfiguration) [S2-4.19] ..... 36**

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

**§H2-6.13 (FSoEBase: Aufrufe der Bibliothek) [S2-4.22] ..... 36**
**§H2-6.14 (FSoEBase: Demomodus nicht möglich) [S2-4.21] ..... 37**
**§H2-6.16 (FSoEBase: Voraussetzungen für Fehleraufdeckung) [S2-4.25] ..... 37**
**§H2-6.17 (FSoEBase: Wiederanlauf) [S2-4.23] ..... 37**

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

## 7.5 Literaturverzeichnis

[N1.1.3] IEC 61131, 2003-1, Programmable controllers, Part 3: Programming languages

[N1.2] IEC 61508: Functional Safety of electric/electronic/programmable electronic safety-related systems, Part 1-7, Edition 2 (2010)DIN/EN/ISO 13849:2008

[N1.3d] DIN EN IEC 62061: Sicherheit von Maschinen

[H1] OEM Integrationshandbuch

[N2.1.1] PLCopen, Technical Committee 5: Safety Software (Technical Specification), Part 1: Concepts and Function Blocks, Version 1.0 (2006-1)

[N3.5.4] Safety over EtherCAT, Protocol specification, Specification, ETG.5100 S (D) V1.2.0

[N3.5.4E] Safety over EtherCAT, Protocol specification, Errata Sheet for ETG.5100 V1.2.0, ETG.5100 ES ® V1.2.0

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

## Änderungshistorie

| | | | |
|---|---|---|---|
| 0.1 | Erstellung | IH | 10.01.2012 |
| 0.2 | Generelle Überarbeitung und Ergänzung der Sicherheitsanforderungen | IH | 12.03.2012 |
| 0.3 | REPEAT Schleifen | SR | 21.03.2012 |
| 0.4 | Version korrigiert | JT | 26.03.2012 |
| 0.5 | Review ok | SR | 26.03.2012 |
| 0.6 | Datentyp BIT hinzugefügt<br><br>Überarbeitung nach TÜV Review 1.1<br><br>ST/LVL: Schachtelungstiefe auf 1 Beschränkt<br>(zur Vermeidung von impliziten Casts) | IH | 08.05.2012 |
| 0.7 | Formulierungen überarbeitet<br><br>Verlinkung auf Systemrequirements komplettiert. | IH | 03.07.2012 |
| 0.8 | Rechtschreibkorrektur | IH | 24.07.2012 |
| 0.9 | Überarbeitung nach Review | IH | 30.07.2012 |
| 0.10 | Verzeichnisse aktualisiert<br>Tippfehler korrigiert | IH | 31.07.2012 |
| 0.11 | Review | MH | 31.07.2012 |
| 1.0 | Released | IH | 31.07.2012 |
| 1.1 | Seit der Überarbeitung in v0.9, waren S2-5.1 und S2-4.1 nicht mehr verlinkt. Die Requirements wurden dementsprechend nachgezogen. | IH | 31.07.2012 |
| 1.2 | Review | MH | 31.07.2012 |
| 2.0 | Released | IH | 31.07.2012 |
| 2.1 | Liste der Sprachen bei der Auflistung in 4.3, sowie Beschreibung in 6.3 korrigiert | IH | 28.09.2012 |
| 2.2 | Beschreibung und Anforderungen für CANopen Safety Master hinzugefügt | IH | 07.12.2012 |
| 2.3 | Review | MH | 17.12.2012 |
| 2.4 | Beschriftungen in Abbildung 2 und 3 neu skaliert | IH | 19.12.2012 |
| 2.5 | Review | MH | 19.12.2012 |
| 3.0 | Released | IH | 19.12.2012 |
| 3.1 | Kapitel 2.3 zur Beschreibung der Projekteinstellungen hinzugefügt | IH | 18.02.2013 |
| 3.2 | Review | MH | 19.03.2013 |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH

| | | | |
|---|---|---|---|
| 4.0 | Release | IH | 22.03.2013 |
| 4.1 | Formatvorlage aktualisiert | IH | 27.09.2013 |
| 4.2 | Neue Safety Anforderungen aus S2 verlinkt | IH | 07.10.2013 |
| 4.3 | Anforderungen für CFC spezifiziert | IH | 17.12.2013 |
| 4.4 | Anforderungen für UML/SC spezifiziert<br>Rechtschreibprüfung durchgeführt<br>Sicherheitsanforderungsnummern fixiert<br>CANopen Safety Anforderungen umbenannt | IH | 18.12.2013 |
| 4.5 | Non-Safe PRGs dokumentiert | IH | 20.12.2013 |
| 4.6 | Neue Regeln, basierend auf der dritten Edition des IEC61131-3 Standards, sowie der neu unterstützten Operatoren AND_THEN und OR_ELSE hinzugefügt. | IH | 02.01.2014 |
| 4.7 | Sprachliche Überarbeitung nach Review | IH | 14.01.2014 |
| 4.8 | EVL / Non-Safe PRG zum Glossar hinzugefügt | IH | 14.01.2014 |
| 4.9 | Review | MH | 15.01.2014 |
| 4.10 | Corrected Requirement Links | IH | 16.01.2014 |
| 4.11 | Review | MH | 21.01.2014 |
| 5.0 | Release | SR | 07.02.2014 |
| 5.1 | Requirement Link korrigiert | SR | 19.08.2014 |
| 5.2 | CODESYS Control SIL2 durch CODESYS Safety SIL2 ersetzt | SR | 15.09.2014 |
| 5.3 | Review | IH | 17.09.2014 |
| 5.4 | Den Begriff Kompilat ersetzt | IH | 18.09.2014 |
| 5.5 | Review | IH | 18.09.2014 |
| 6.0 | Release | SR | 18.09.2014 |
| 6.1 | FSoE Anforderungen aus [S2] verlinkt | MH | 01.10.2021 |
| 6.2 | Review | LS | 04.10.2021 |
| 6.3 | Rework after review | MH | 04.10.2021 |
| 6.4 | Rework after review | MH | 05:10:2021 |
| 6.5 | Review | LS | 05.10.2021 |
| 7.0 | Release | MH | 26.04.2022 |
| 7.1 | FSoE Anforderungen S2-4.23, S2-4.24 und S2-4.25 aus [S2] verlinkt und beschrieben. | MH | 01.06.2022 |
| 7.2 | Review | MHu | 02.06.2022 |
| 8.0 | Review | MH | 02.06.2022 |

Template: Vorlage_tecdoc_V1.0.dotx

© 3S-Smart Software Solutions GmbH